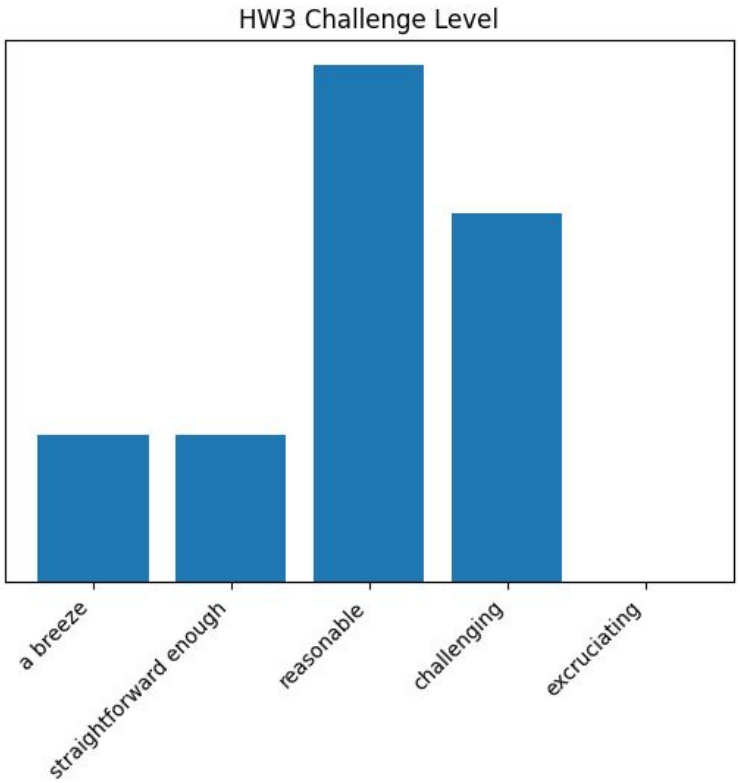
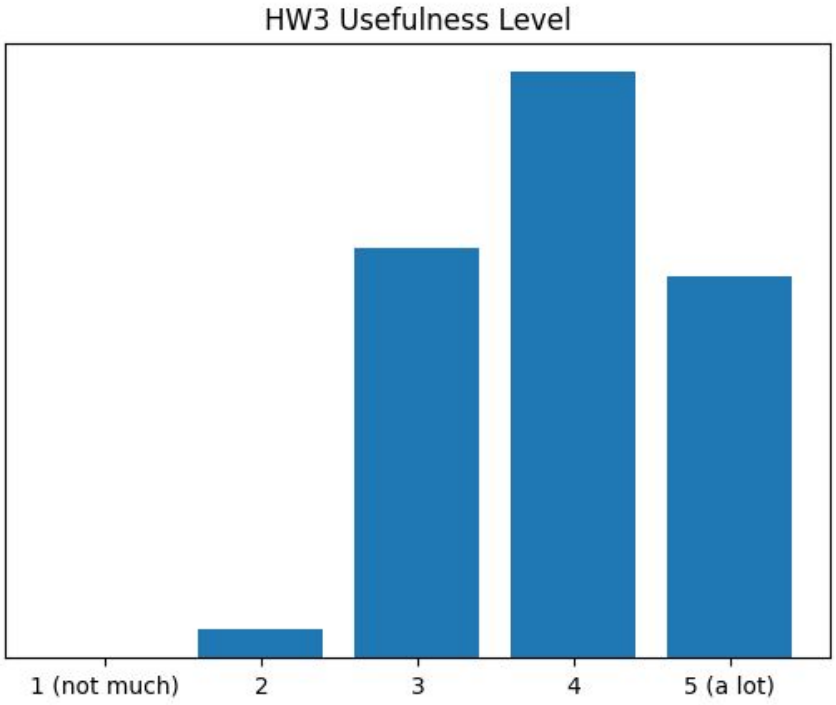
CSE 493L Section 5

Data Structures in Rust

Looking Back on Homework 3





Homework 5: Due Dates

- **Monday, February 9th:** test case - hard deadline
- **Wednesday, February 11th**: code, questions - 2 day grace period

Implementing single-threaded `insert` and `get` on a Red-Black Tree!

Check-in

- Before we get started: *any questions?*

Also make sure everyone has a handout!

Aside: Packed vs Linked Data Structures

Packed data structures *tend to be** more performant than linked ones.

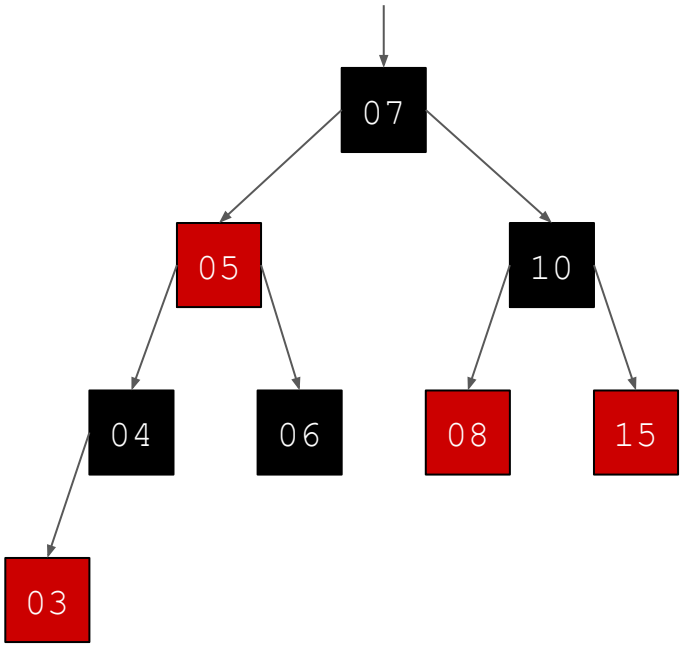
- **Vector** is more performant than **LinkedList**
- **HashMap** is more performant than **TreeMap**

Why?

* with some exceptions (BTrees, link-based graphs, and other use-case specific things)

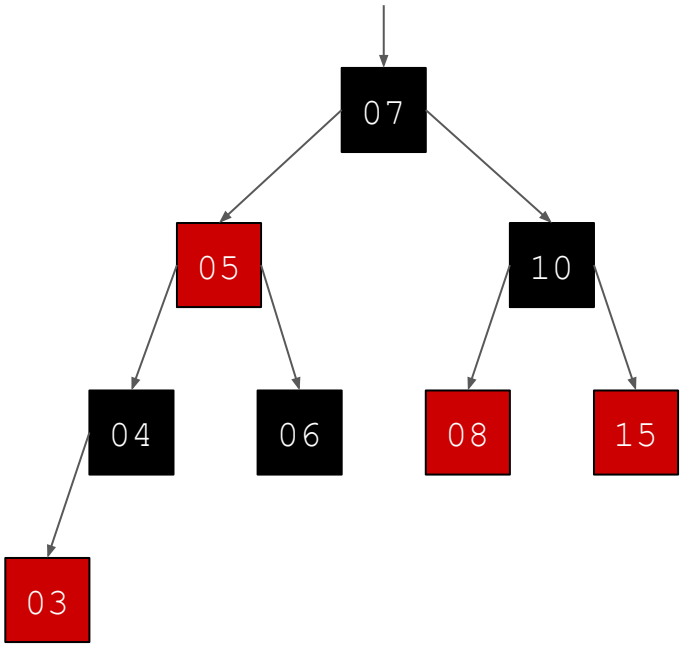
Red-Black Trees

- Balanced BST
  - `O(log(n))` get, insert, delete
- Similar to AVL
- Each node is colored R/B
  - Just one extra bit per node!
  - Maintain three invariants regarding coloring to keep tree balanced
- HW5: Implement RBTree to act as a KVStore.
- Really nice ownership structure!



Red-Black Trees

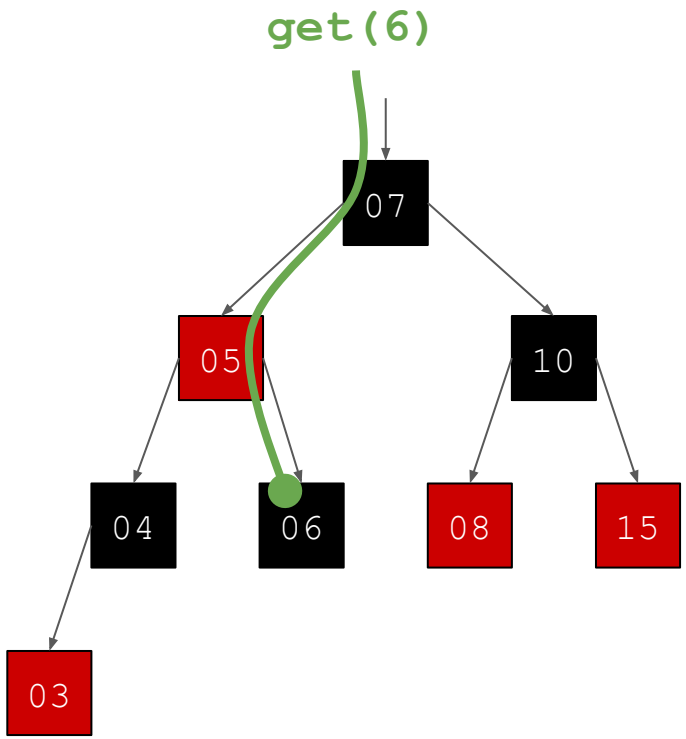
- Balanced BST
  - `O(log(n))` get, insert, delete
- Similar to AVL
- Each node is colored R/B
  - Just one extra bit per node!
  - Maintain three invariants regarding coloring to keep tree balanced
- HW5: Implement RBTree to act as a KVStore.
- Really nice ownership structure!



How do we get a value from this tree?

Red-Black Trees

- Balanced BST
  - `O(log(n))` get, insert, delete
- Similar to AVL
- Each node is colored R/B
  - Just one extra bit per node!
  - Maintain three invariants regarding coloring to keep tree balanced
- HW5: Implement RBTree to act as a KVStore.
- Really nice ownership structure!

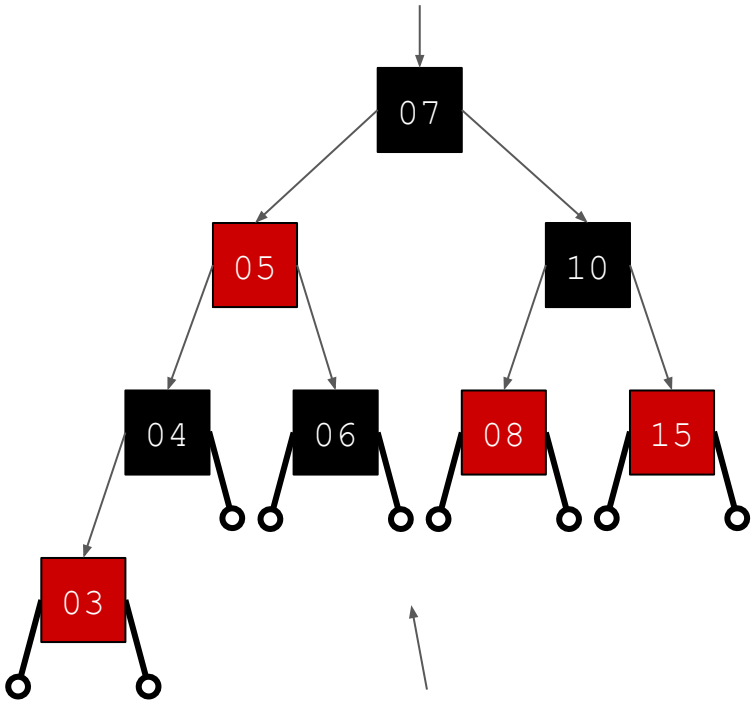


How do we get a value from this tree? *The same way as a regular BST!*

Red-Black Trees: *Insert*

The three invariants that balance the tree:

- **Every node is either red or black. The root node is black. Null nodes are black.**

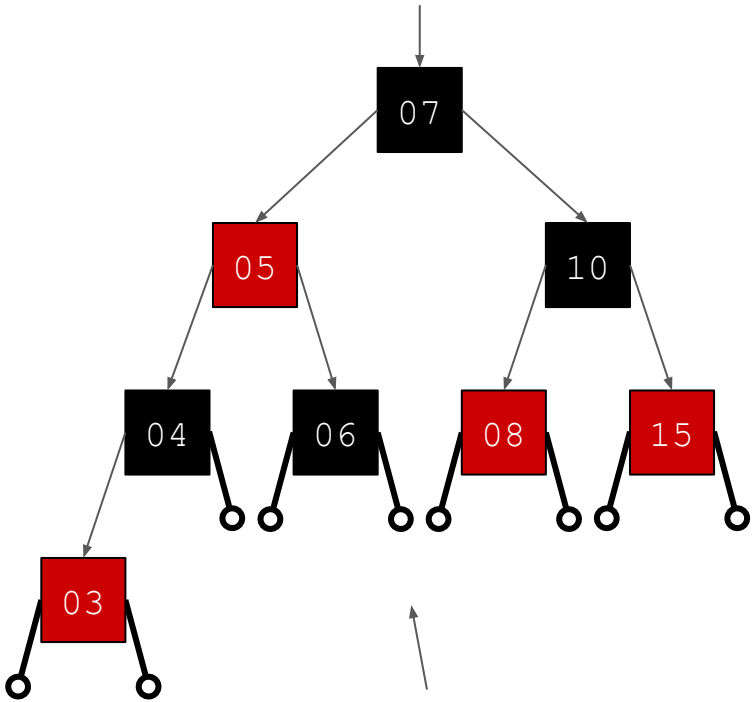


Let's draw the null nodes too!

Red-Black Trees: *Insert*

The three invariants that balance the tree:

- **Every node is either red or black. The root node is black. Null nodes are black.**



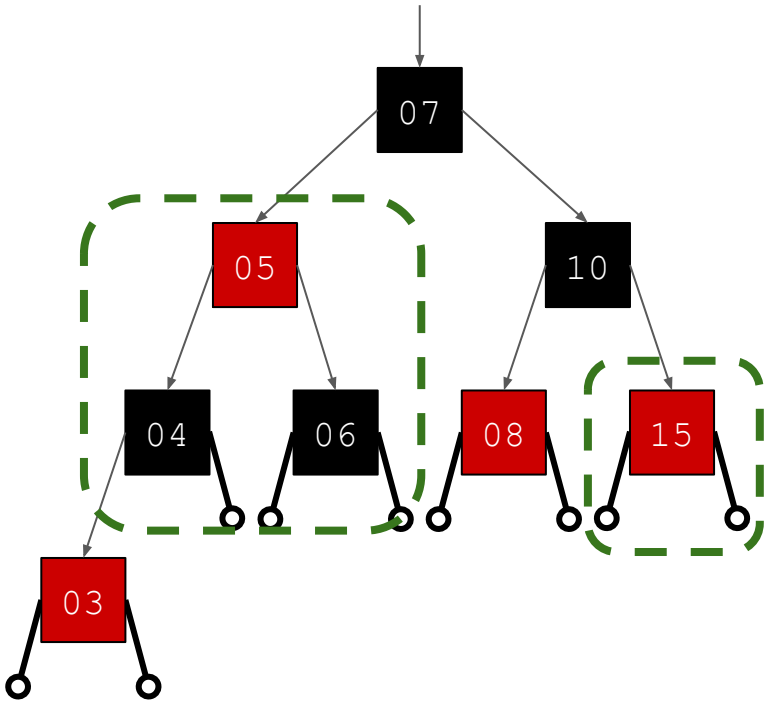
Let's draw the null nodes too!

Generally, fixed by setting the root node to black at the end of `insert`

Red-Black Trees: *Insert*

The three invariants that balance the tree:

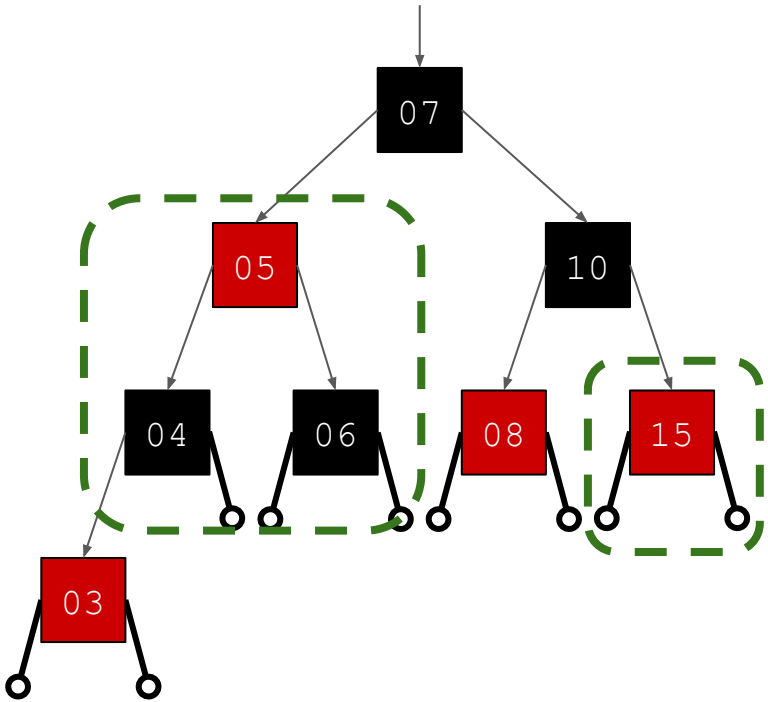
- Every node is either red or black. The root node is black. Null nodes are black.
- **The children of a red node must both be black nodes.**



Red-Black Trees: *Insert*

The three invariants that balance the tree:

- Every node is either red or black. The root node is black. Null nodes are black.
- **The children of a red node must both be black nodes.**

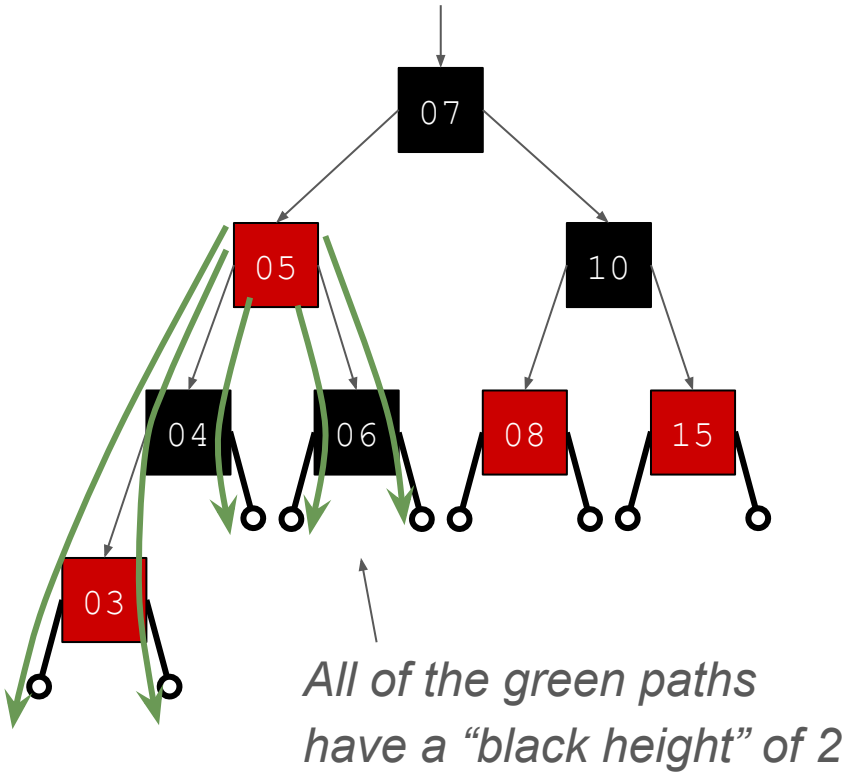


This imposes a rule on how R/B nodes are distributed across levels

Red-Black Trees: *Insert*

The three invariants that balance the tree:

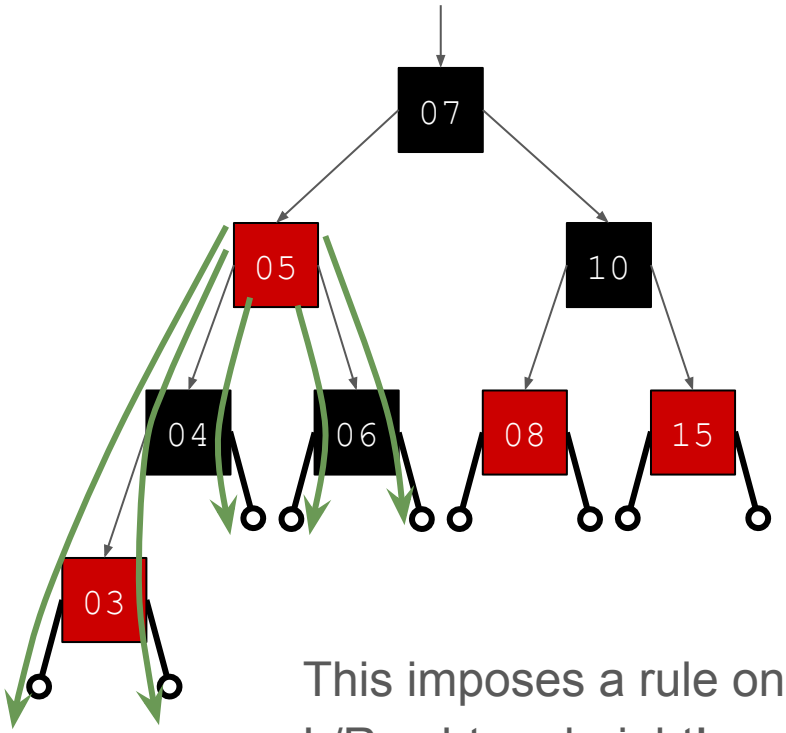
- Every node is either red or black. The root node is black. Null nodes are black.
- The children of a red node must both be black nodes.
- **For each node, all paths from that node to any leaf in its subtree have the same number of black nodes, counting the null nodes.**



Red-Black Trees: *Insert*

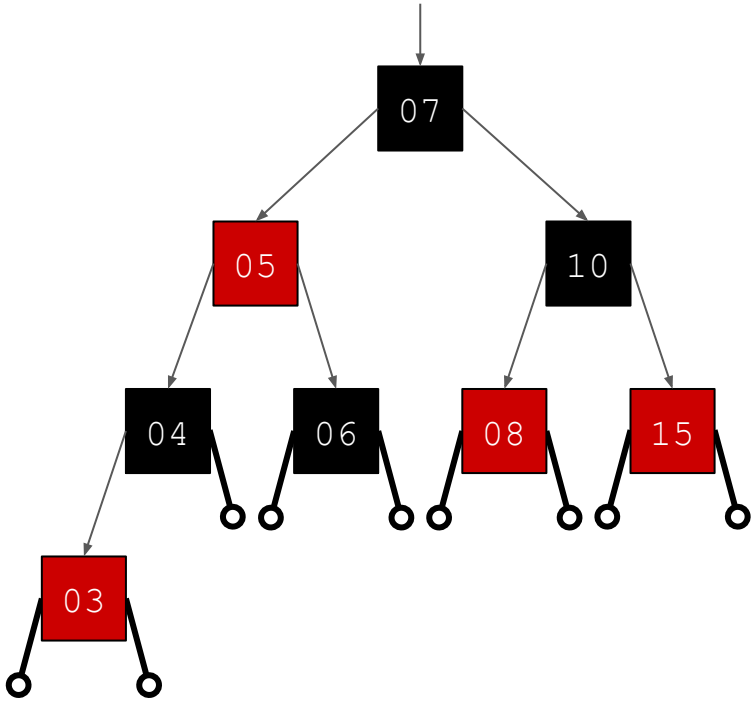
The three invariants that balance the tree:

- Every node is either red or black. The root node is black. Null nodes are black.
- The children of a red node must both be black nodes.
- **For each node, all paths from that node to any leaf in its subtree have the same number of black nodes, counting the null nodes.**



This imposes a rule on L/R subtree height!

Red-Black Trees: *Insert*



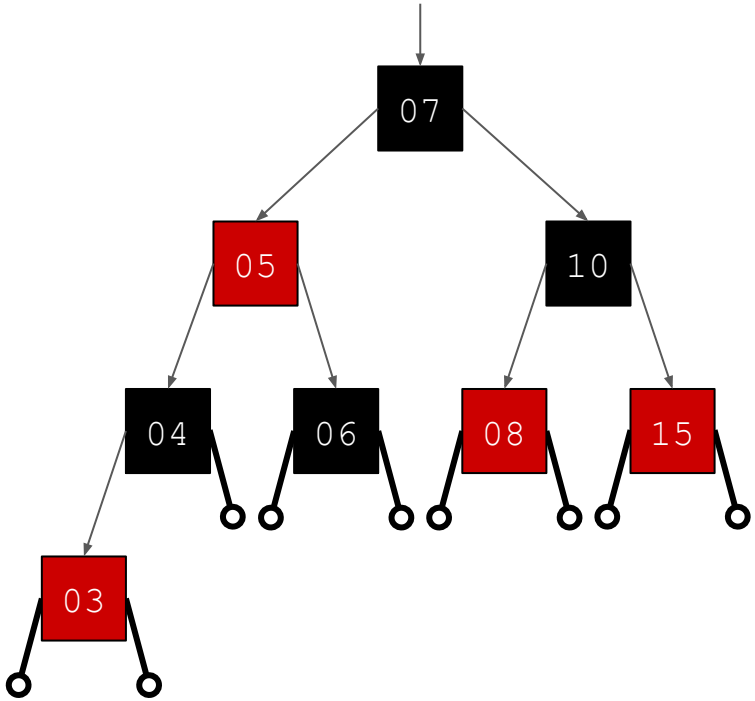
Invariants:

- Every node is either red or black. The root node is black. Null nodes are black.
- The children of a red node must both be black nodes.
- For each node, all paths from that node to any leaf in its subtree have the same number of black nodes counting the null nodes.

General insertion pseudocode:

1. Perform a standard BST insert (recursively)
2. Detect possible violations while backtracking
3. Fix violations as necessary with rotations and recolors

Red-Black Trees: *Insert*



Invariants:

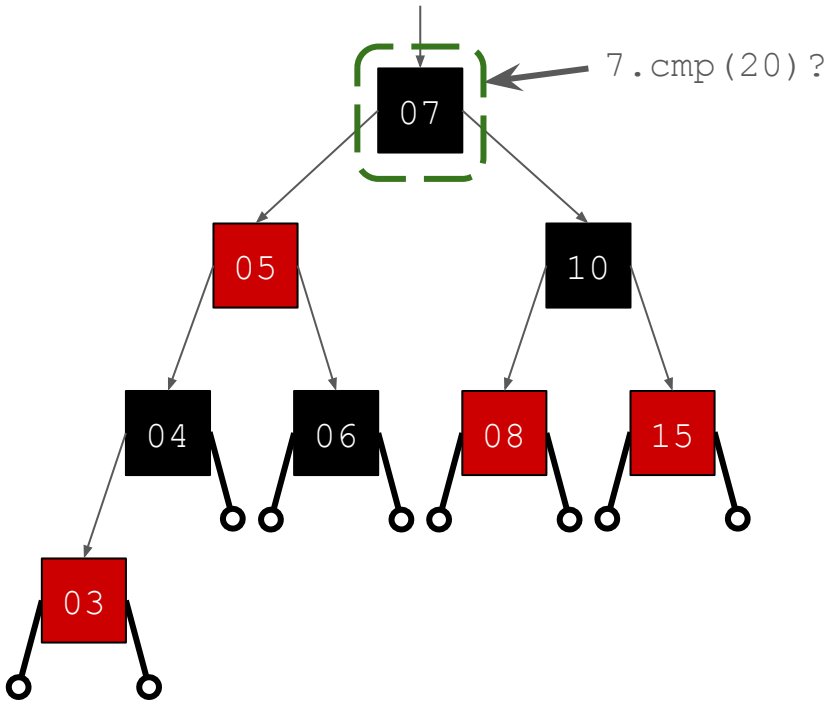
- Every node is either red or black. The root node is black. Null nodes are black.
- The children of a red node must both be black nodes.
- For each node, all paths from that node to any leaf in its subtree have the same number of black nodes counting the null nodes.

General insertion pseudocode:

1. Perform a standard BST insert (recursively)
2. Detect possible violations while backtracking
3. Fix violations as necessary with rotations and recolors

Let's perform `insert(20)`!

Red-Black Trees: *Insert*



Invariants:

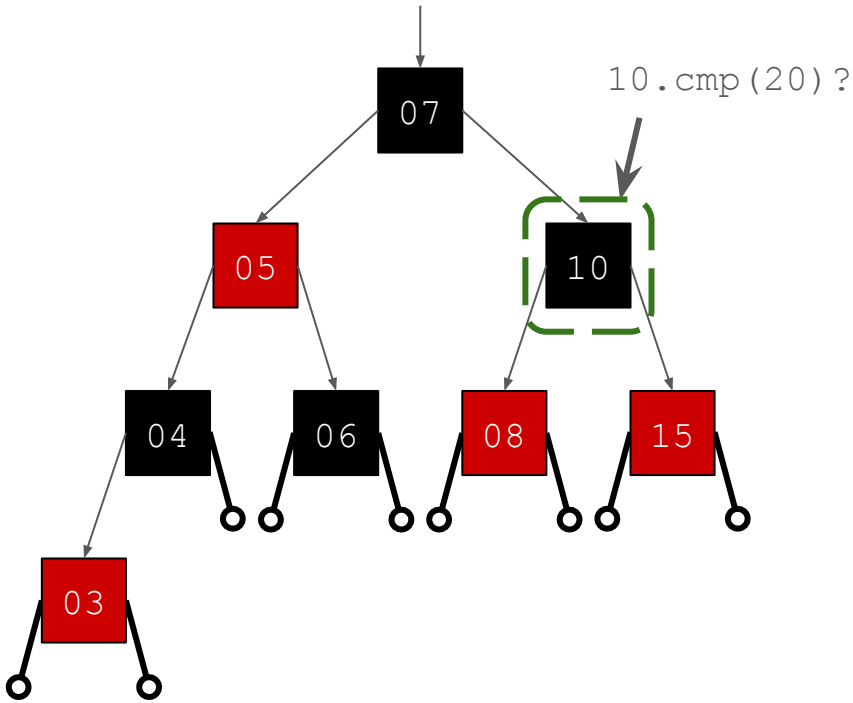
- Every node is either red or black. The root node is black. Null nodes are black.
- The children of a red node must both be black nodes.
- For each node, all paths from that node to any leaf in its subtree have the same number of black nodes counting the null nodes.

General insertion pseudocode:

1. Perform a standard BST insert (recursively)
2. Detect possible violations while backtracking
3. Fix violations as necessary with rotations and recolors

Let's perform `insert(20)`!

Red-Black Trees: *Insert*



Invariants:

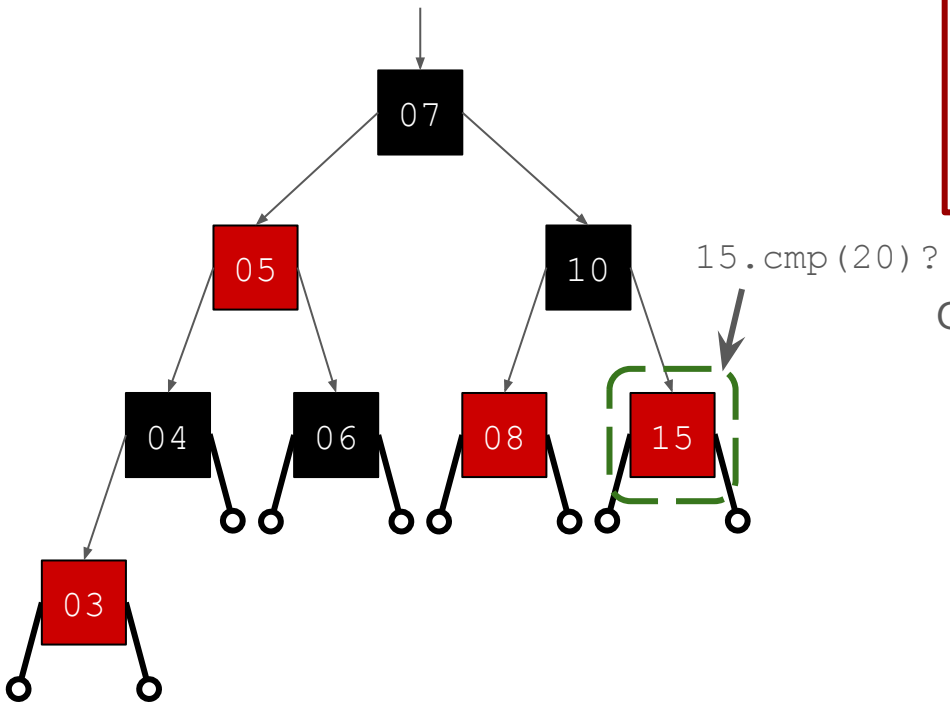
- Every node is either red or black. The root node is black. Null nodes are black.
- The children of a red node must both be black nodes.
- For each node, all paths from that node to any leaf in its subtree have the same number of black nodes counting the null nodes.

General insertion pseudocode:

1. Perform a standard BST insert (recursively)
2. Detect possible violations while backtracking
3. Fix violations as necessary with rotations and recolors

Let's perform `insert(20)`!

Red-Black Trees: *Insert*



Invariants:

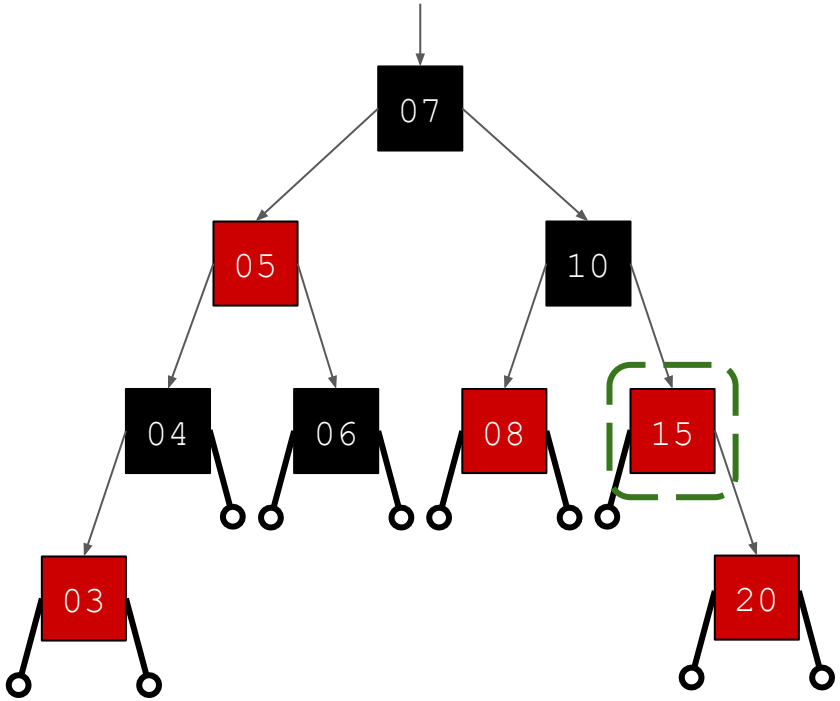
- Every node is either red or black. The root node is black. Null nodes are black.
- The children of a red node must both be black nodes.
- For each node, all paths from that node to any leaf in its subtree have the same number of black nodes counting the null nodes.

General insertion pseudocode:

1. Perform a standard BST insert (recursively)
2. Detect possible violations while backtracking
3. Fix violations as necessary with rotations and recolors

Let's perform `insert(20)`!

Red-Black Trees: *Insert*



Invariants:

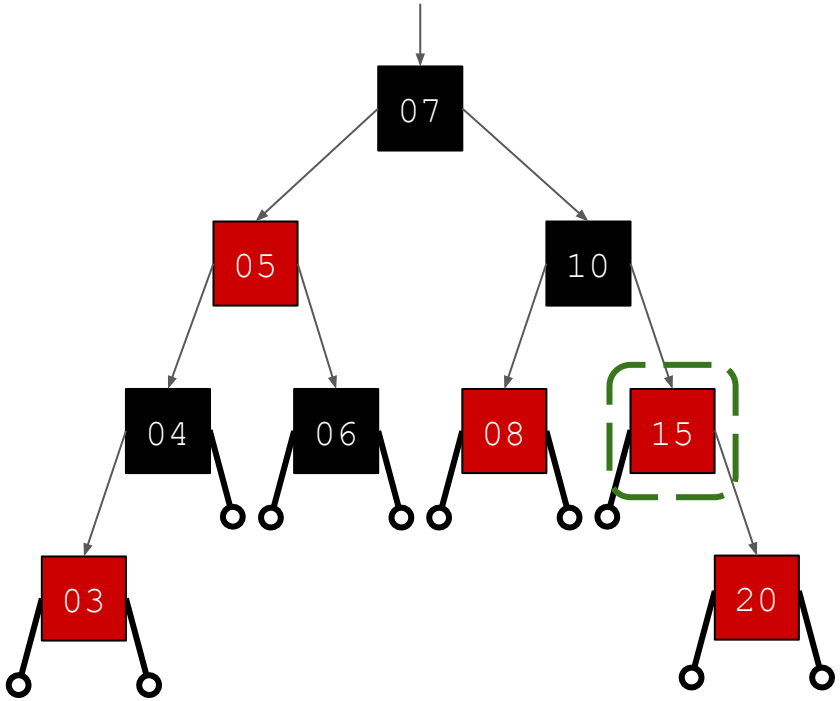
- Every node is either red or black. The root node is black. Null nodes are black.
- The children of a red node must both be black nodes.
- For each node, all paths from that node to any leaf in its subtree have the same number of black nodes counting the null nodes.

General insertion pseudocode:

1. Perform a standard BST insert (recursively)
2. Detect possible violations while backtracking
3. Fix violations as necessary with rotations and recolors

Newly inserted nodes are *always* red!

Red-Black Trees: *Insert*



Invariants:

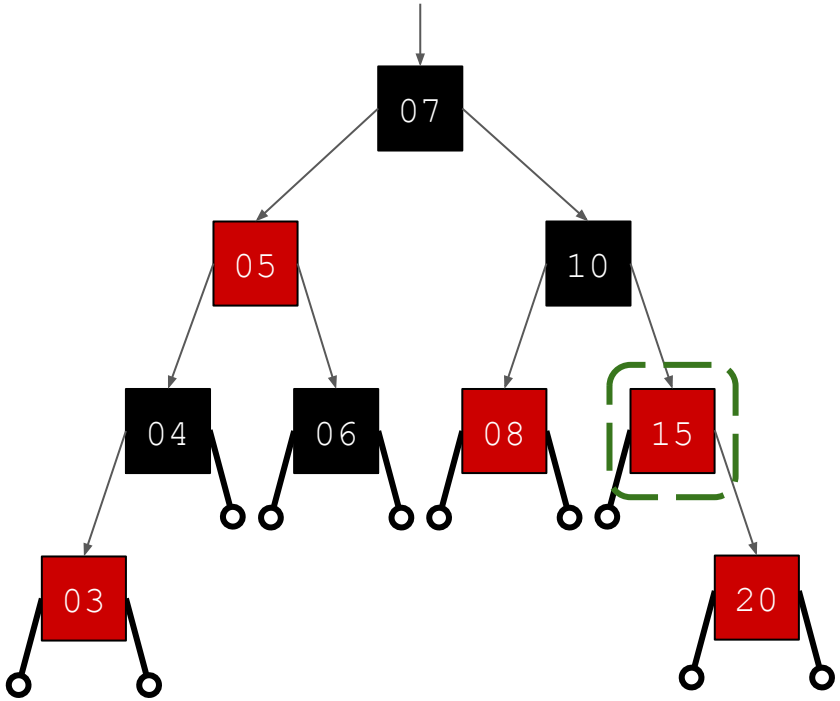
- Every node is either red or black. The root node is black. Null nodes are black.
- The children of a red node must both be black nodes.
- For each node, all paths from that node to any leaf in its subtree have the same number of black nodes counting the null nodes.

General insertion pseudocode:

1. Perform a standard BST insert (recursively)
2. Detect possible violations while backtracking
3. Fix violations as necessary with rotations and recolors

Are there any violations in the current frame?

Red-Black Trees: *Insert*



Invariants:

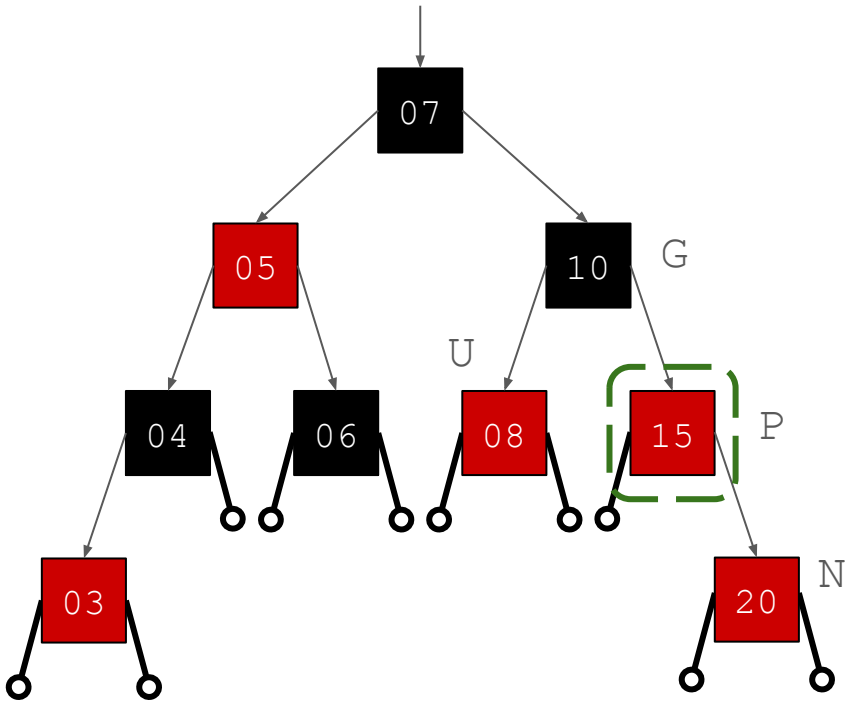
- Every node is either red or black. The root node is black. Null nodes are black.
- The children of a red node must both be black nodes.
- For each node, all paths from that node to any leaf in its subtree have the same number of black nodes counting the null nodes.

General insertion pseudocode:

1. Perform a standard BST insert (recursively)
2. Detect possible violations while backtracking
3. Fix violations as necessary with rotations and recolors

Are there any violations in the current frame?
Violation of inv. 2!

Red-Black Trees: *Insert*



Invariants:

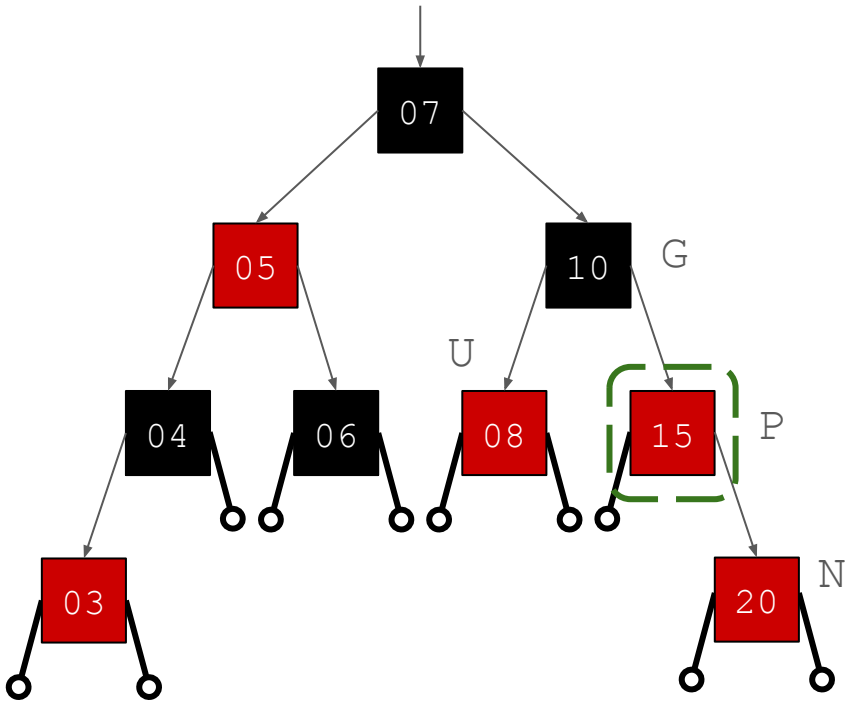
- Every node is either red or black. The root node is black. Null nodes are black.
- The children of a red node must both be black nodes.
- For each node, all paths from that node to any leaf in its subtree have the same number of black nodes counting the null nodes.

General insertion pseudocode:

1. Perform a standard BST insert (recursively)
2. Detect possible violations while backtracking
3. Fix violations as necessary with rotations and recolors

Let's put some labels on each relevant node to give us some language to talk about the violation

Red-Black Trees: *Insert*



Invariants:

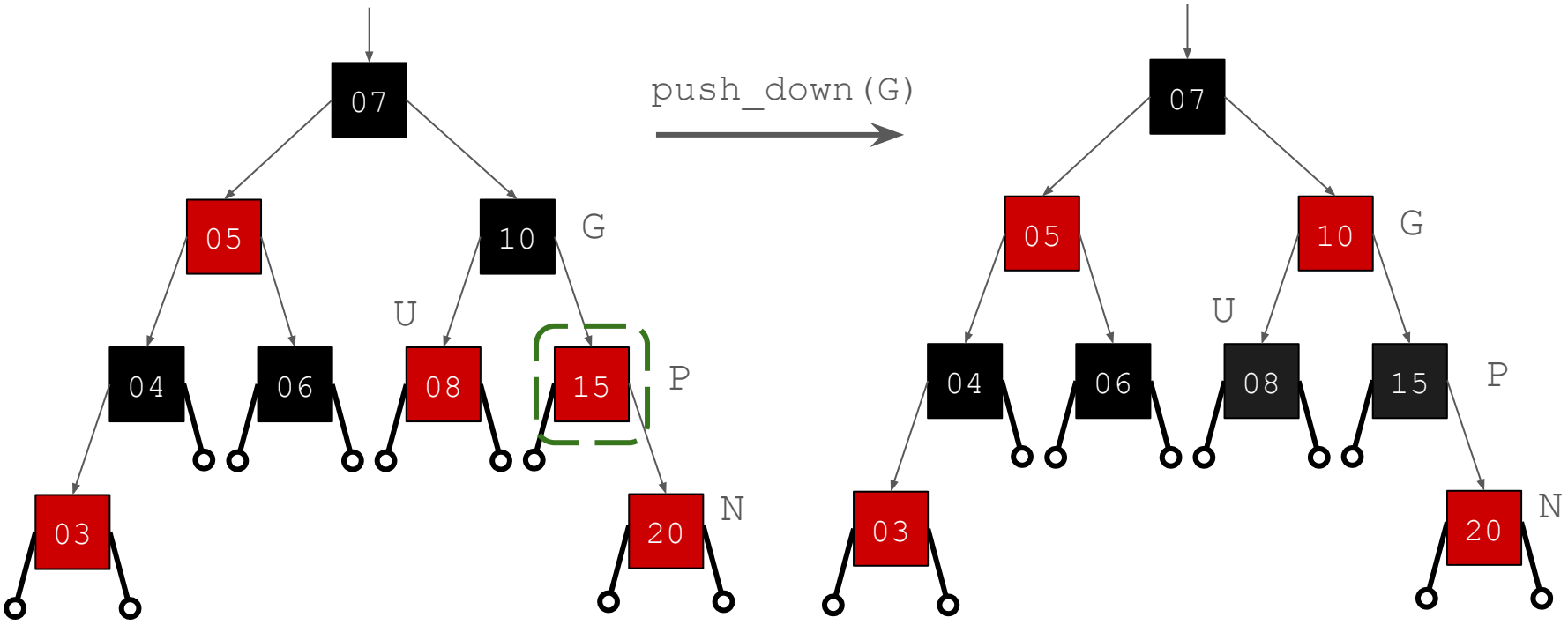
- Every node is either red or black. The root node is black. Null nodes are black.
- The children of a red node must both be black nodes.
- For each node, all paths from that node to any leaf in its subtree have the same number of black nodes counting the null nodes.

General insertion pseudocode:

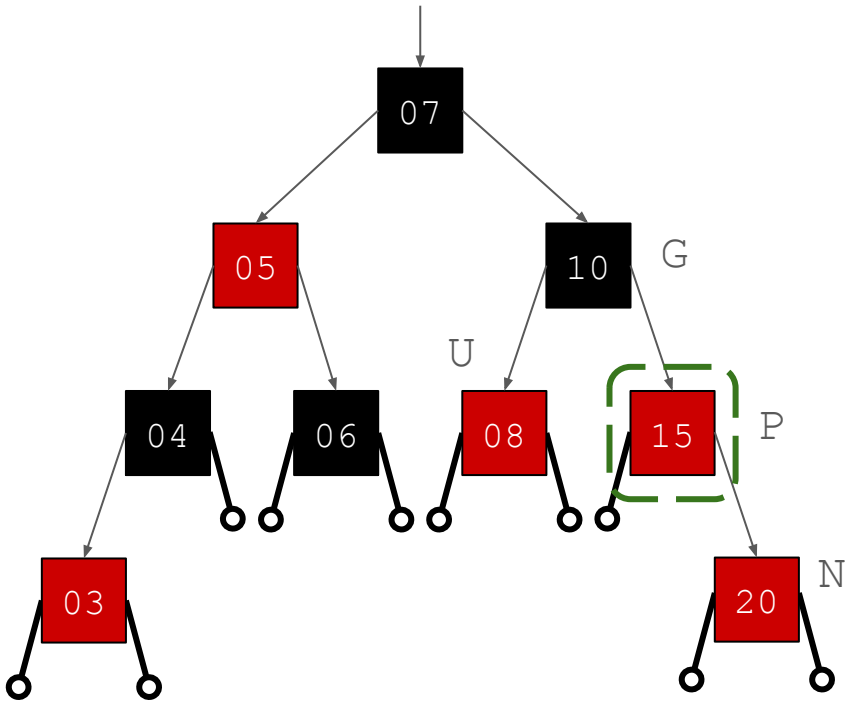
1. Perform a standard BST insert (recursively)
2. Detect possible violations while backtracking
3. Fix violations as necessary with rotations and recolors

Because U is red, we can “push down” G’s blackness to fix inv. 2!

Red-Black Trees: *Insert*



Red-Black Trees: *Insert*



Invariants:

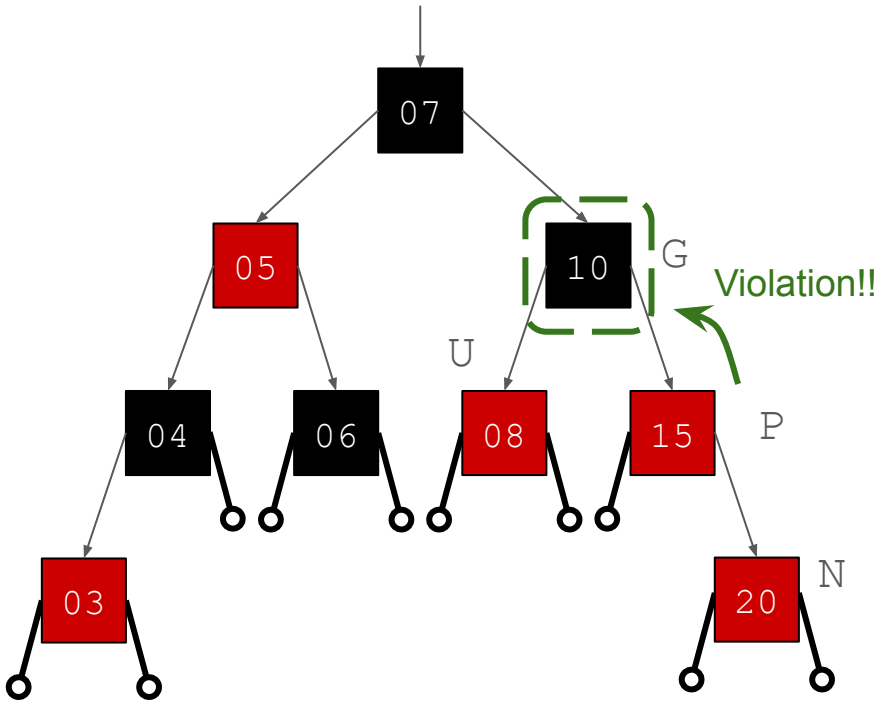
- Every node is either red or black. The root node is black. Null nodes are black.
- The children of a red node must both be black nodes.
- For each node, all paths from that node to any leaf in its subtree have the same number of black nodes counting the null nodes.

General insertion pseudocode:

1. Perform a standard BST insert (recursively)
2. Detect possible violations while backtracking
3. Fix violations as necessary with rotations and recolors

Wait, to do that, we need information about U! That's outside of the current frame…

Red-Black Trees: *Insert*



Invariants:

- Every node is either red or black. The root node is black. Null nodes are black.
- The children of a red node must both be black nodes.
- For each node, all paths from that node to any leaf in its subtree have the same number of black nodes counting the null nodes.

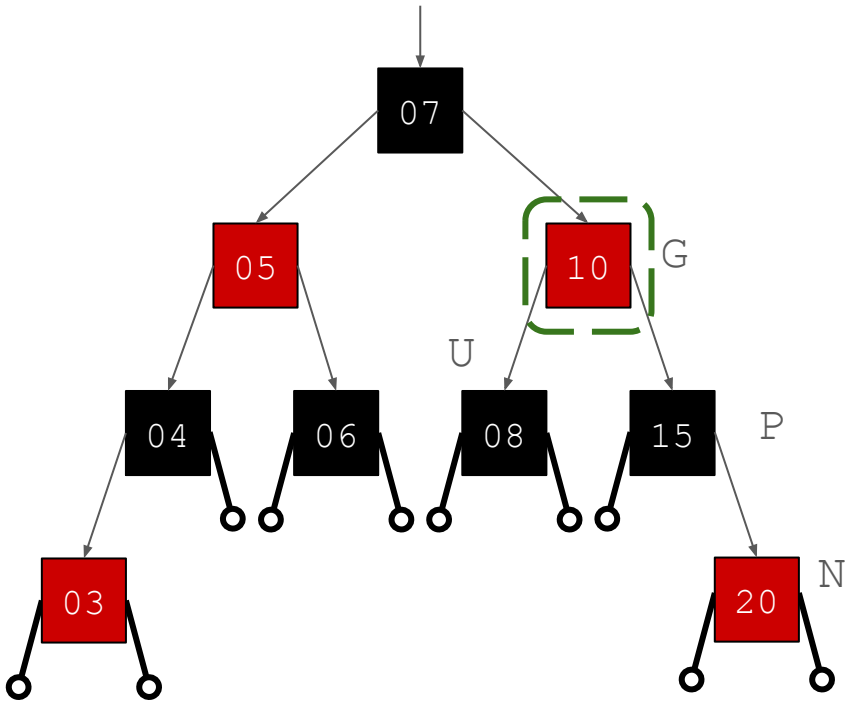
General insertion pseudocode:

1. Perform a standard BST insert (recursively)
2. Detect possible violations while backtracking
3. Fix violations as necessary with rotations and recolors

Wait, to do that, we need information about `U`*!*
That's outside of the current frame…

- We need to relay that a violation occurred to `G`...

Red-Black Trees: *Insert*



Invariants:

- Every node is either red or black. The root node is black. Null nodes are black.
- The children of a red node must both be black nodes.
- For each node, all paths from that node to any leaf in its subtree have the same number of black nodes counting the null nodes.

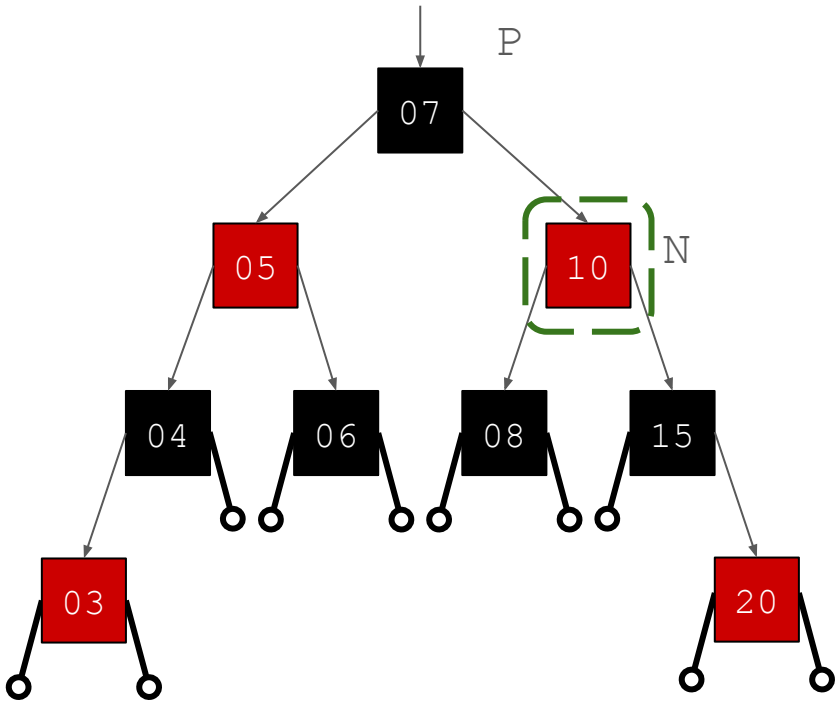
General insertion pseudocode:

1. Perform a standard BST insert (recursively)
2. Detect possible violations while backtracking
3. Fix violations as necessary with rotations and recolors

`G` finds out that `U` is red, and performs `push_down`

Are we done?

Red-Black Trees: *Insert*



Invariants:

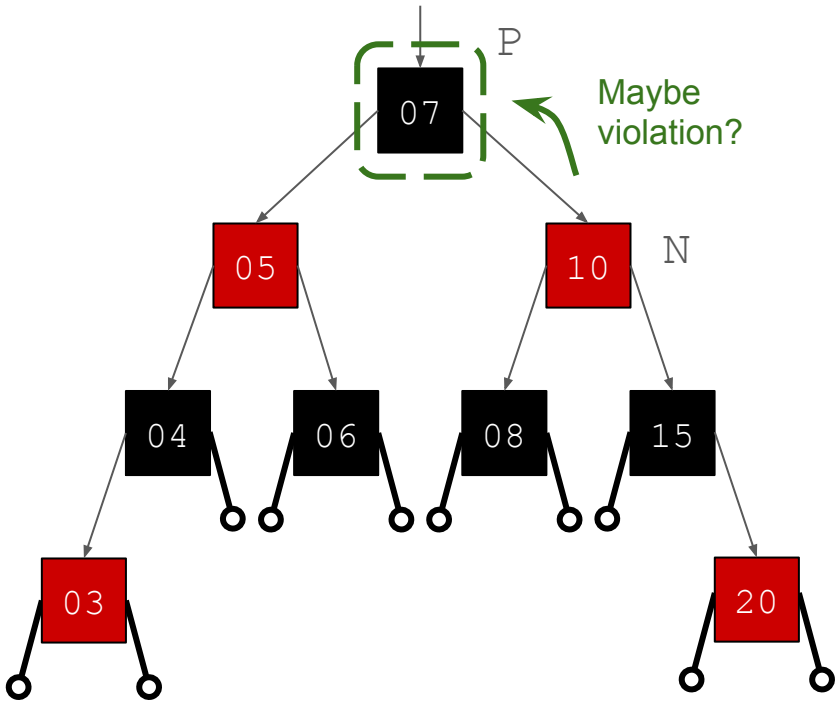
- Every node is either red or black. The root node is black. Null nodes are black.
- The children of a red node must both be black nodes.
- For each node, all paths from that node to any leaf in its subtree have the same number of black nodes counting the null nodes.

General insertion pseudocode:

1. Perform a standard BST insert (recursively)
2. Detect possible violations while backtracking
3. Fix violations as necessary with rotations and recolors

Changing G to red effectively "inserts" a new red node at G. Depending on G's parent, we could have another violation!

Red-Black Trees: *Insert*



Invariants:

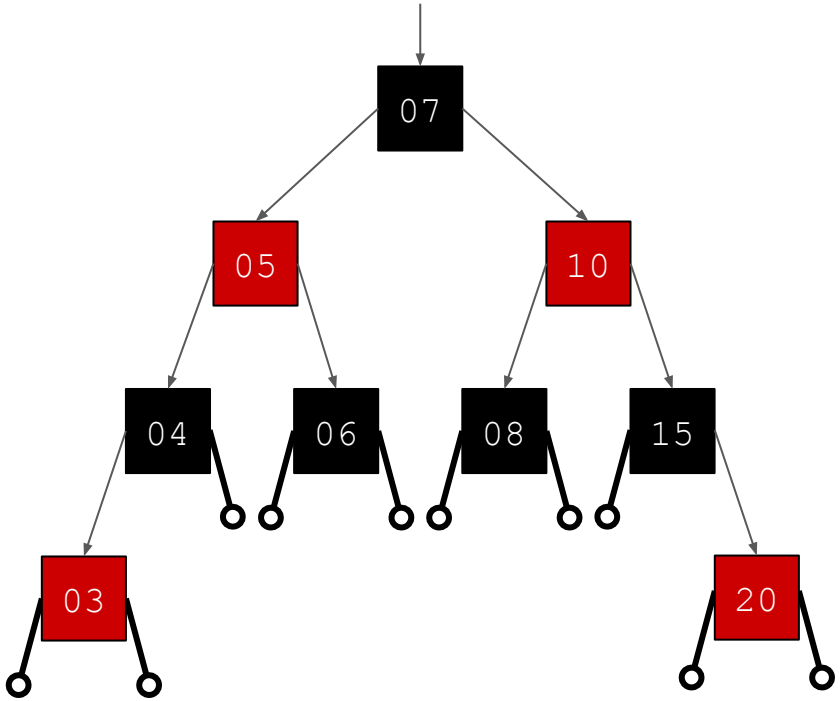
- Every node is either red or black. The root node is black. Null nodes are black.
- The children of a red node must both be black nodes.
- For each node, all paths from that node to any leaf in its subtree have the same number of black nodes counting the null nodes.

General insertion pseudocode:

1. Perform a standard BST insert (recursively)
2. Detect possible violations while backtracking
3. Fix violations as necessary with rotations and recolors

New P can determine if it's an actual violation, and restart this process

Red-Black Trees: *Insert*



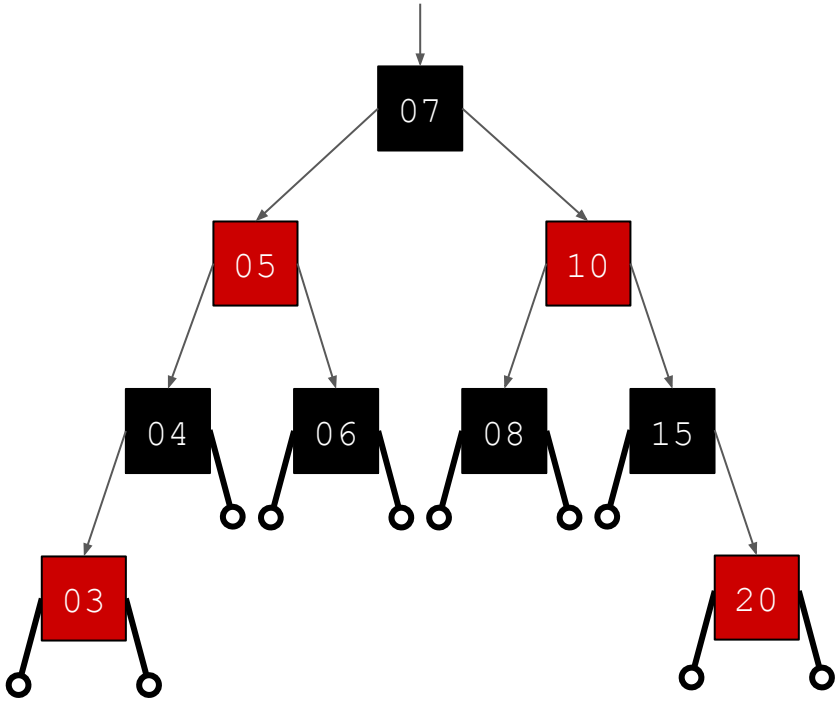
Invariants:

- Every node is either red or black. The root node is black. Null nodes are black.
- The children of a red node must both be black nodes.
- For each node, all paths from that node to any leaf in its subtree have the same number of black nodes counting the null nodes.

General insertion pseudocode:

1. Perform a standard BST insert (recursively)
2. Detect possible violations while backtracking
3. Fix violations as necessary with rotations and recolors

In this case, there is no violation at this level, so we are done!



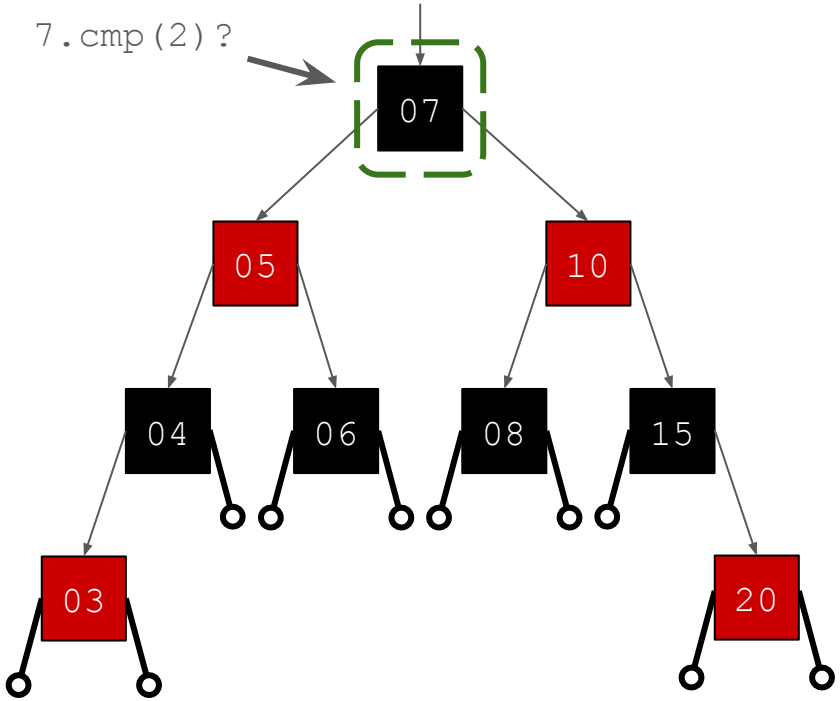
Invariants:

- Every node is either red or black. The root node is black. Null nodes are black.
- The children of a red node must both be black nodes.
- For each node, all paths from that node to any leaf in its subtree have the same number of black nodes counting the null nodes.

General insertion pseudocode:

1. Perform a standard BST insert (recursively)
2. Detect possible violations while backtracking
3. Fix violations as necessary with rotations and recolors

Let's now do `insert(2)`



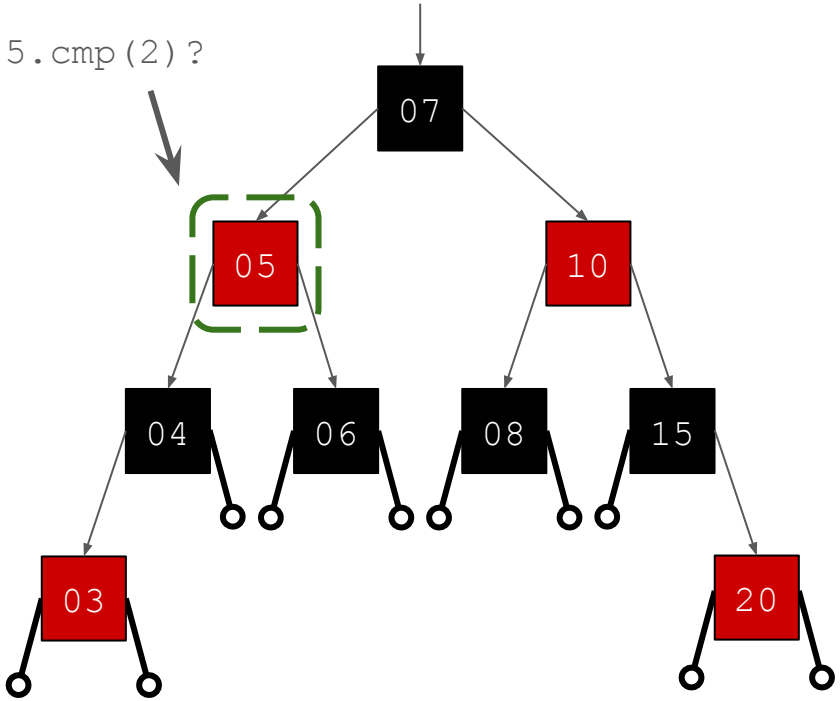
Invariants:

- Every node is either red or black. The root node is black. Null nodes are black.
- The children of a red node must both be black nodes.
- For each node, all paths from that node to any leaf in its subtree have the same number of black nodes counting the null nodes.

General insertion pseudocode:

1. Perform a standard BST insert (recursively)
2. Detect possible violations while backtracking
3. Fix violations as necessary with rotations and recolors

Let's now do `insert(2)`



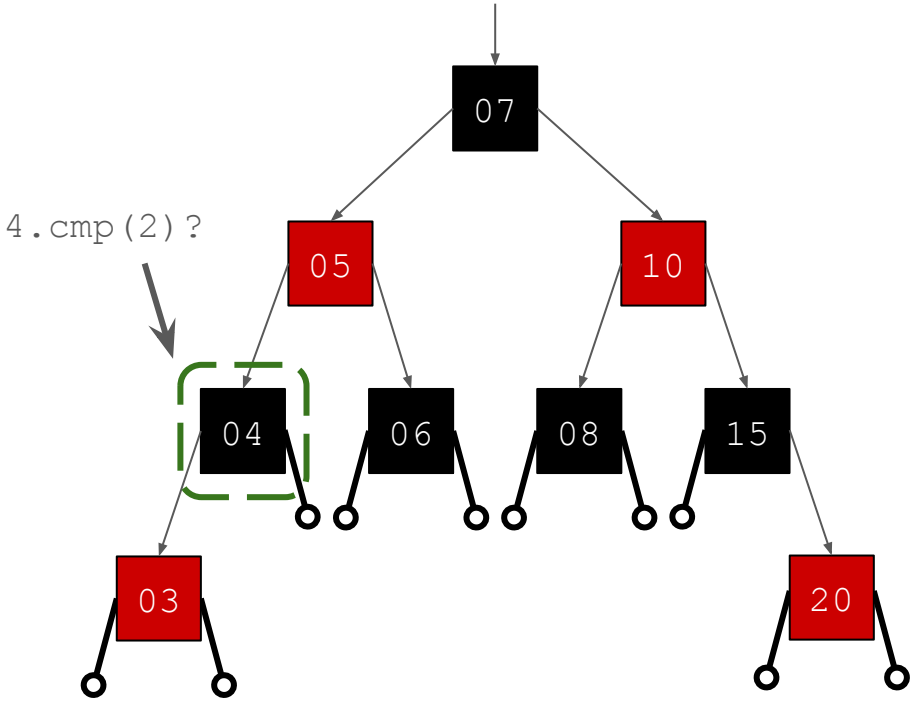
Invariants:

- Every node is either red or black. The root node is black. Null nodes are black.
- The children of a red node must both be black nodes.
- For each node, all paths from that node to any leaf in its subtree have the same number of black nodes counting the null nodes.

General insertion pseudocode:

1. Perform a standard BST insert (recursively)
2. Detect possible violations while backtracking
3. Fix violations as necessary with rotations and recolors

Let's now do `insert(2)`



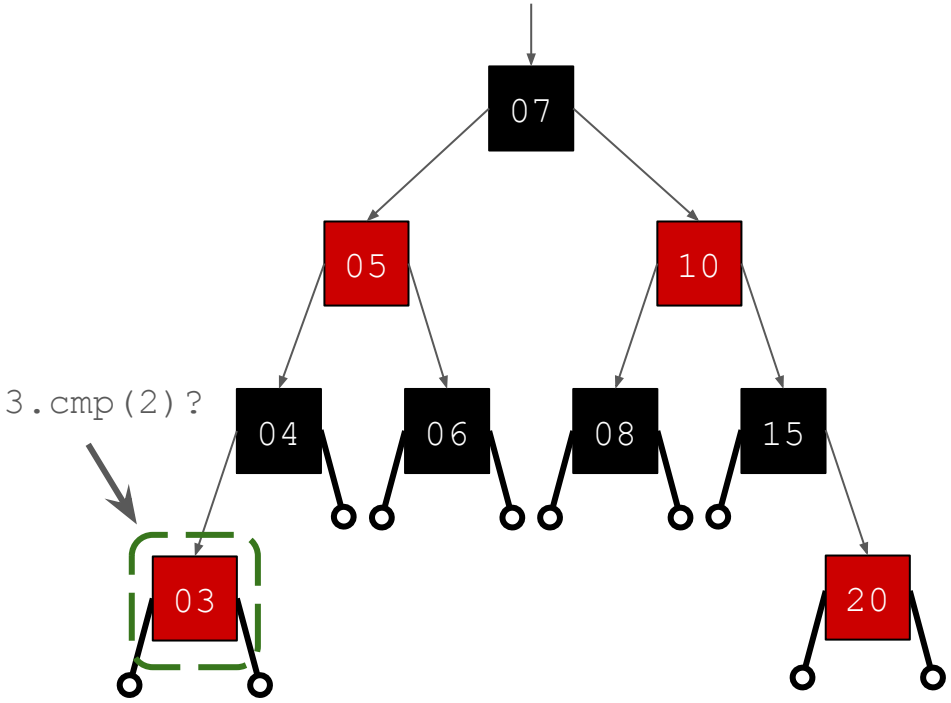
Invariants:

- Every node is either red or black. The root node is black. Null nodes are black.
- The children of a red node must both be black nodes.
- For each node, all paths from that node to any leaf in its subtree have the same number of black nodes counting the null nodes.

General insertion pseudocode:

1. Perform a standard BST insert (recursively)
2. Detect possible violations while backtracking
3. Fix violations as necessary with rotations and recolors

Let's now do `insert(2)`



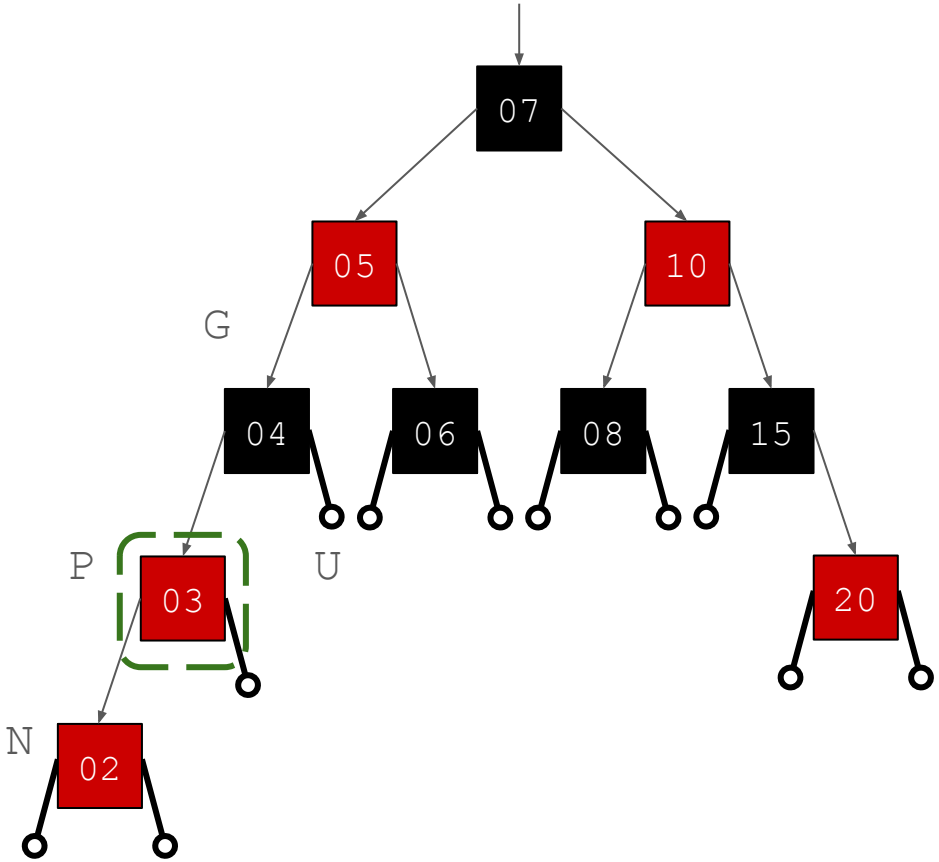
Invariants:

- Every node is either red or black. The root node is black. Null nodes are black.
- The children of a red node must both be black nodes.
- For each node, all paths from that node to any leaf in its subtree have the same number of black nodes counting the null nodes.

General insertion pseudocode:

1. Perform a standard BST insert (recursively)
2. Detect possible violations while backtracking
3. Fix violations as necessary with rotations and recolors

Let's now do `insert(2)`



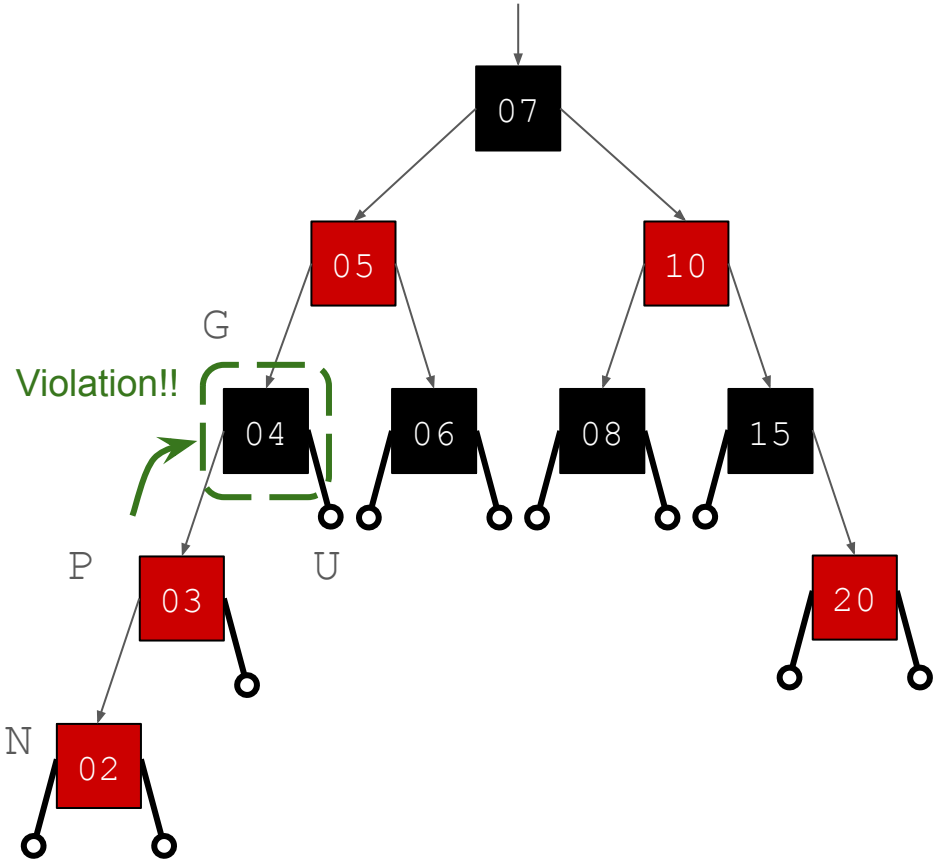
Invariants:

- Every node is either red or black. The root node is black. Null nodes are black.
- The children of a red node must both be black nodes.
- For each node, all paths from that node to any leaf in its subtree have the same number of black nodes counting the null nodes.

General insertion pseudocode:

1. Perform a standard BST insert (recursively)
2. Detect possible violations while backtracking
3. Fix violations as necessary with rotations and recolors

Again, we violate inv. 2…



Invariants:

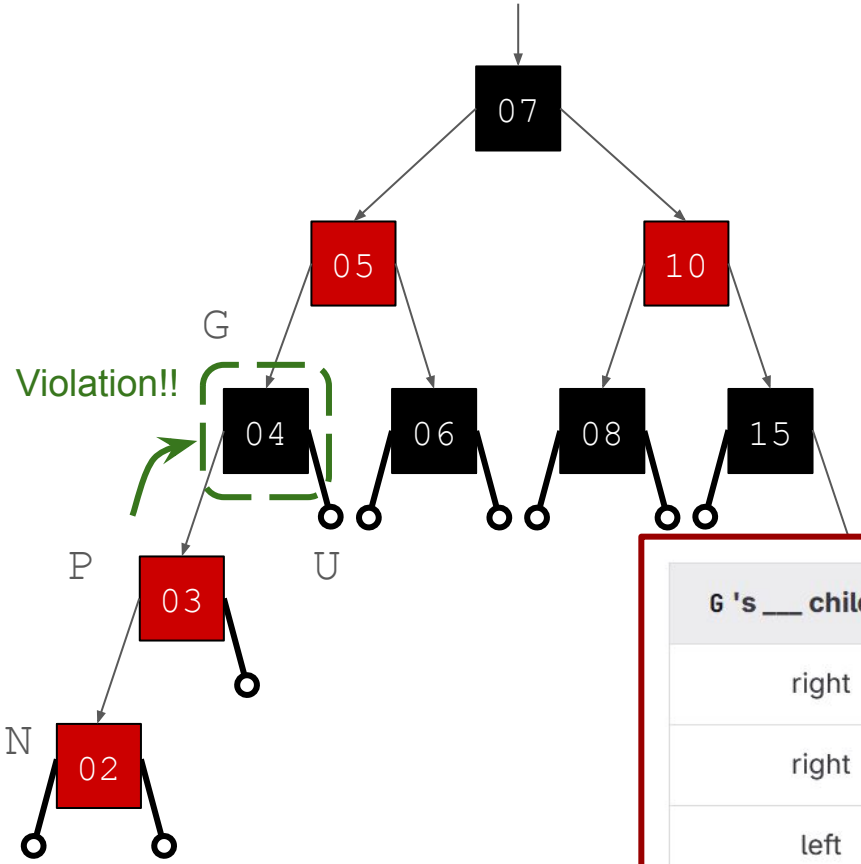
- Every node is either red or black. The root node is black. Null nodes are black.
- The children of a red node must both be black nodes.
- For each node, all paths from that node to any leaf in its subtree have the same number of black nodes counting the null nodes.

General insertion pseudocode:

1. Perform a standard BST insert (recursively)
2. Detect possible violations while backtracking
3. Fix violations as necessary with rotations and recolors

`U` is black – we can't just `push_down`

Fix tree with rotations and recolors



Invariants:

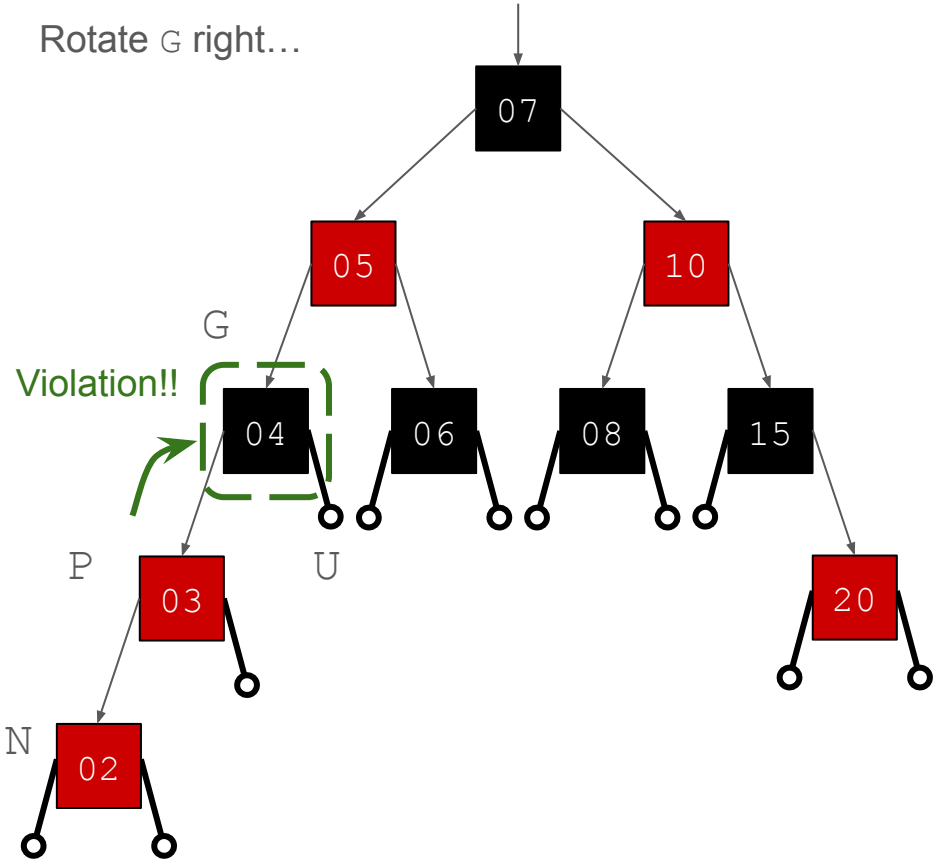
- Every node is either red or black. The root node is black. Null nodes are black.
- The children of a red node must both be black nodes.
- For each node, all paths from that node to any leaf in its subtree have the same number of black nodes counting the null nodes.

General insertion pseudocode:

1. Perform a standard BST insert (recursively)

G 's ___ child is P	P 's ___ child is N	Rotate	Swap Colors
right	right	G left	G and P
right	left	P right, then G left	G and N
left	right	P left, then G right	G and N
left	left	G right	G and P

Rotate G right…



Invariants:

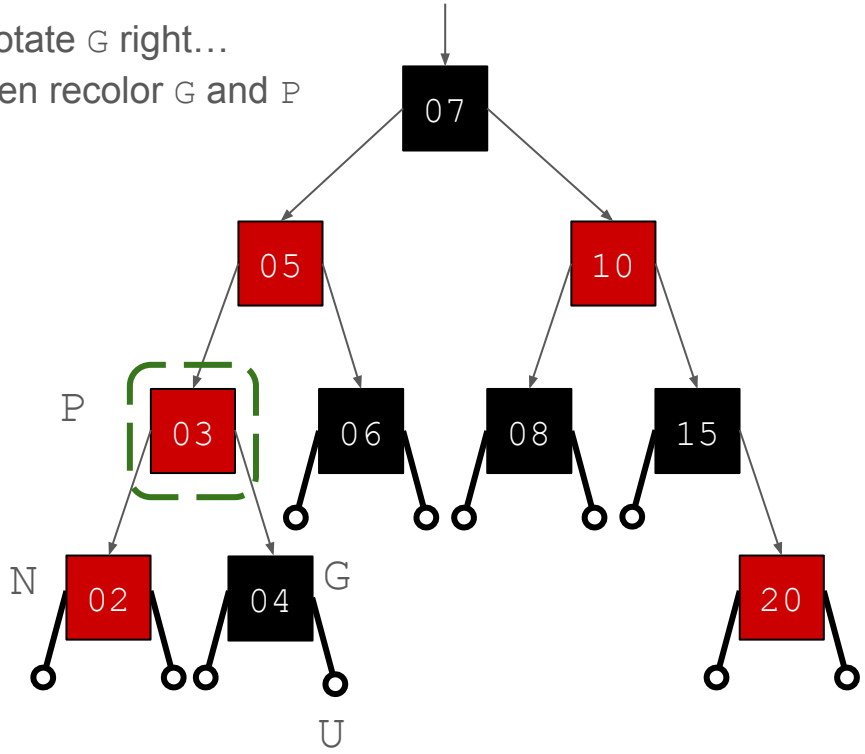
- Every node is either red or black. The root node is black. Null nodes are black.
- The children of a red node must both be black nodes.
- For each node, all paths from that node to any leaf in its subtree have the same number of black nodes counting the null nodes.

General insertion pseudocode:

1. Perform a standard BST insert (recursively)
2. Detect possible violations while backtracking
3. Fix violations as necessary with rotations and recolors

Because of the L/R relationships of G, P, and N, we need to rotate G right, then swap G and P's color

Rotate G right…
then recolor G and P



Invariants:

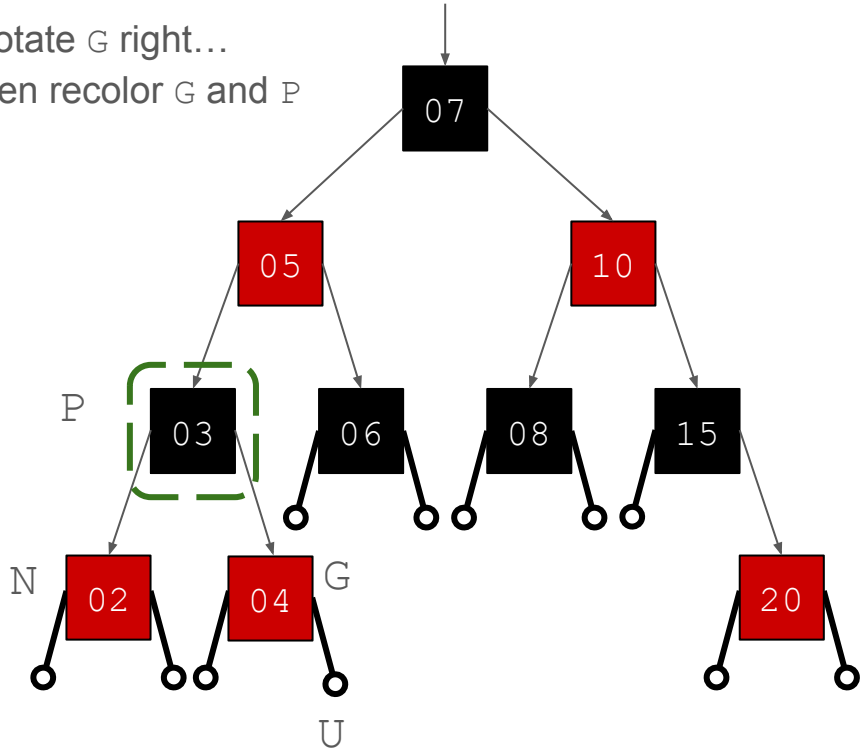
- Every node is either red or black. The root node is black. Null nodes are black.
- The children of a red node must both be black nodes.
- For each node, all paths from that node to any leaf in its subtree have the same number of black nodes counting the null nodes.

General insertion pseudocode:

1. Perform a standard BST insert (recursively)
2. Detect possible violations while backtracking
3. Fix violations as necessary with rotations and recolors

Because of the L/R relationships of G, P, and N, we need to rotate G right, then swap G and P's color

Rotate G right…
then recolor G and P



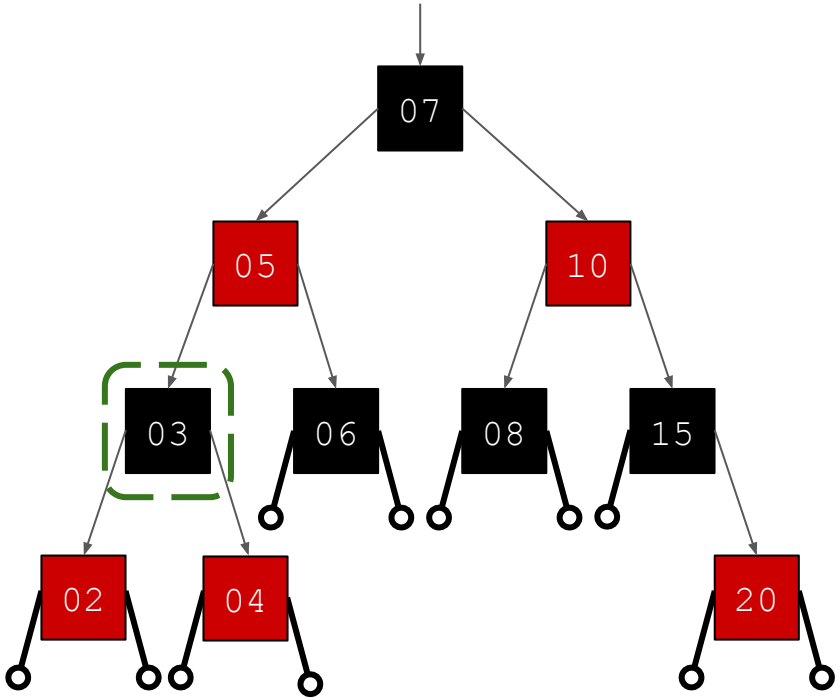
Invariants:

- Every node is either red or black. The root node is black. Null nodes are black.
- The children of a red node must both be black nodes.
- For each node, all paths from that node to any leaf in its subtree have the same number of black nodes counting the null nodes.

General insertion pseudocode:

1. Perform a standard BST insert (recursively)
2. Detect possible violations while backtracking
3. Fix violations as necessary with rotations and recolors

Because of the L/R relationships of G, P, and N, we need to rotate G right, then swap G and P's color



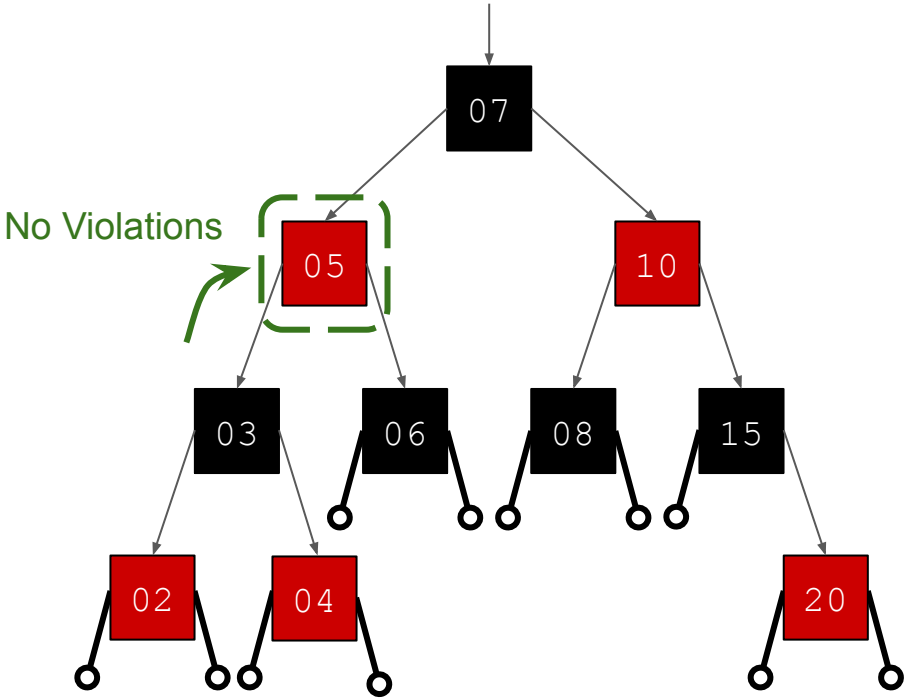
Invariants:

- Every node is either red or black. The root node is black. Null nodes are black.
- The children of a red node must both be black nodes.
- For each node, all paths from that node to any leaf in its subtree have the same number of black nodes counting the null nodes.

General insertion pseudocode:

1. Perform a standard BST insert (recursively)
2. Detect possible violations while backtracking
3. Fix violations as necessary with rotations and recolors

Current frame ends up with a black node, so there are no more violations!



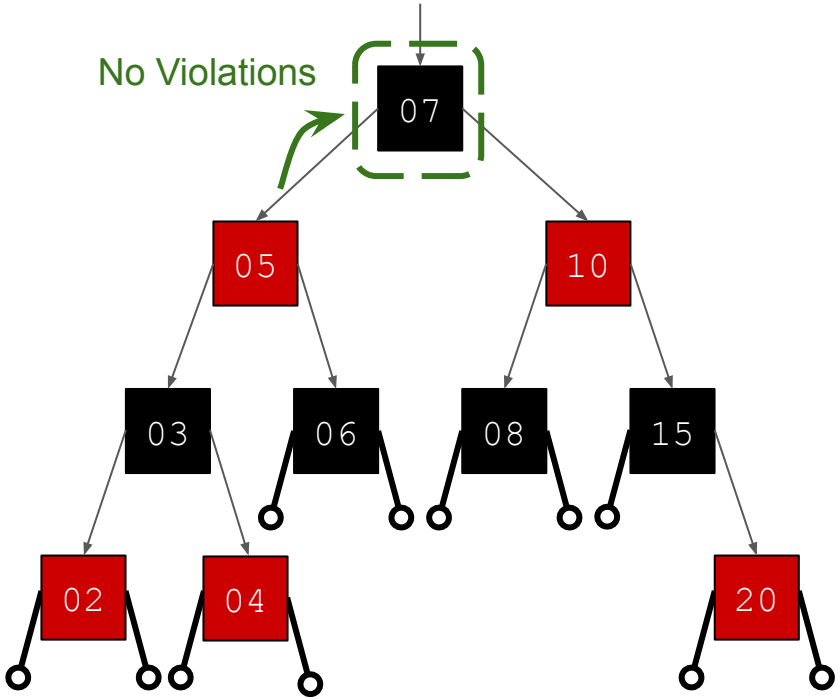
Invariants:

- Every node is either red or black. The root node is black. Null nodes are black.
- The children of a red node must both be black nodes.
- For each node, all paths from that node to any leaf in its subtree have the same number of black nodes counting the null nodes.

General insertion pseudocode:

1. Perform a standard BST insert (recursively)
2. Detect possible violations while backtracking
3. Fix violations as necessary with rotations and recolors

Current frame ends up with a black node, so there are no more violations!



Invariants:

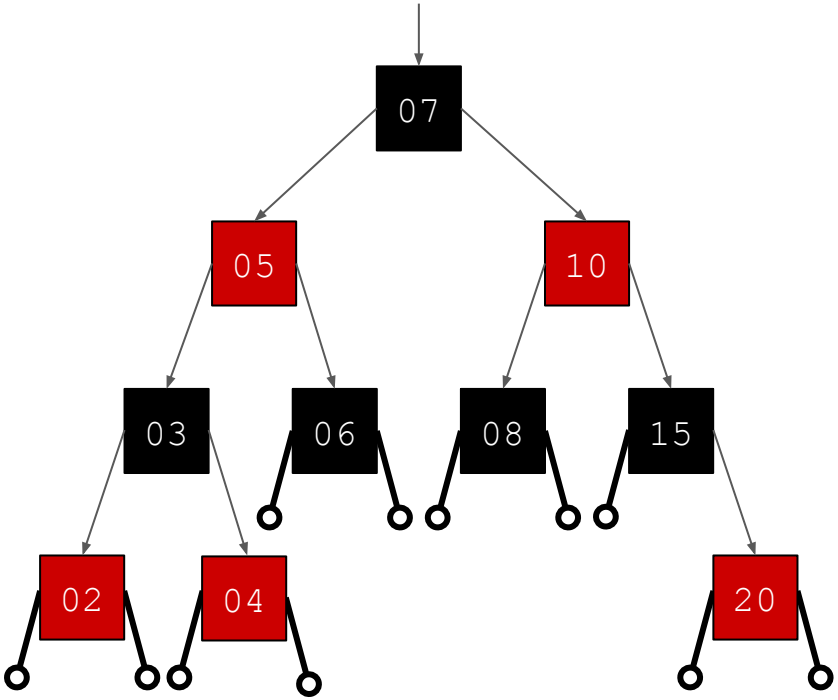
- Every node is either red or black. The root node is black. Null nodes are black.
- The children of a red node must both be black nodes.
- For each node, all paths from that node to any leaf in its subtree have the same number of black nodes counting the null nodes.

General insertion pseudocode:

1. Perform a standard BST insert (recursively)
2. Detect possible violations while backtracking
3. Fix violations as necessary with rotations and recolors

Current frame ends up with a black node, so there are no more violations!

Red-Black Trees: *Insert*



Invariants:

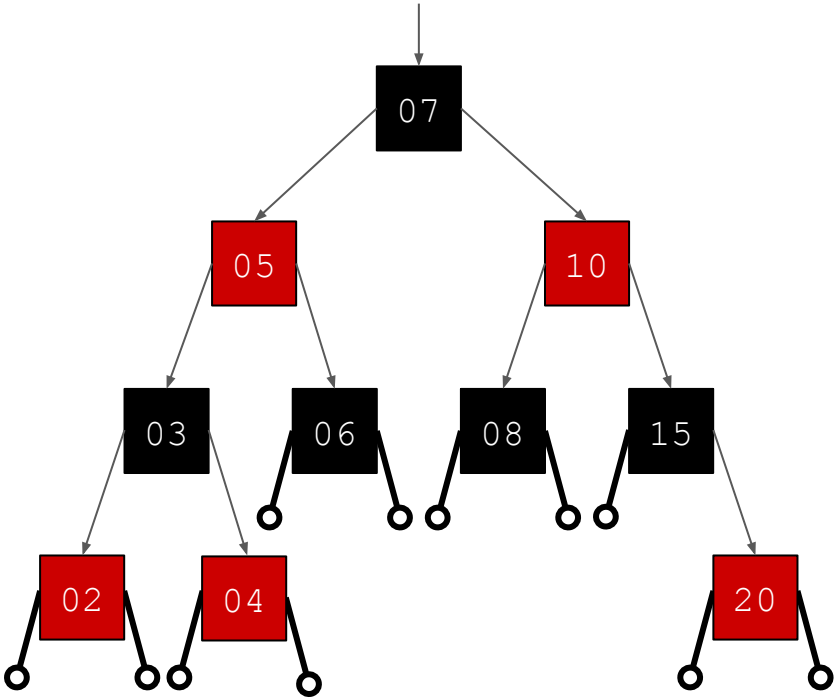
- Every node is either red or black. The root node is black. Null nodes are black.
- The children of a red node must both be black nodes.
- For each node, all paths from that node to any leaf in its subtree have the same number of black nodes counting the null nodes.

General insertion pseudocode:

1. Perform a standard BST insert (recursively)
2. Detect possible violations while backtracking
3. Fix violations as necessary with rotations and recolors

Our tree is balanced again!

Red-Black Trees: *Insert*



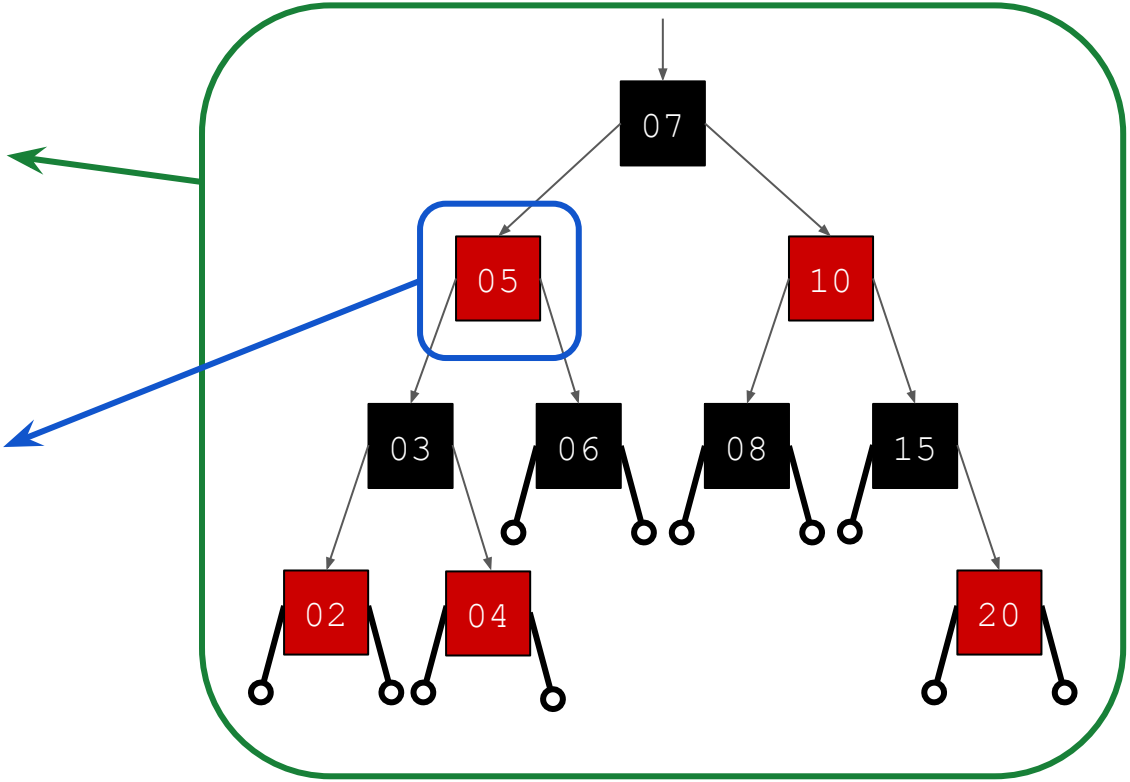
More things to keep in mind:

- Carefully consider what the return type is from any recursive `insert()` call
  - Encode all information necessary to be able to decide how to fix the tree
- At the end of insertion, if the root node changes to red, change it to black
- If the insertion is for a key that already exists, `RBTree.insert()` should replace it, and return the old value

Red-Black Trees: *Rust-Specific Stuff*

```rust
pub struct RBTree<K, V>
where
    K: Ord,
{
    root: Option<Box<Node<K, V>>>,
    size: usize,
}
```

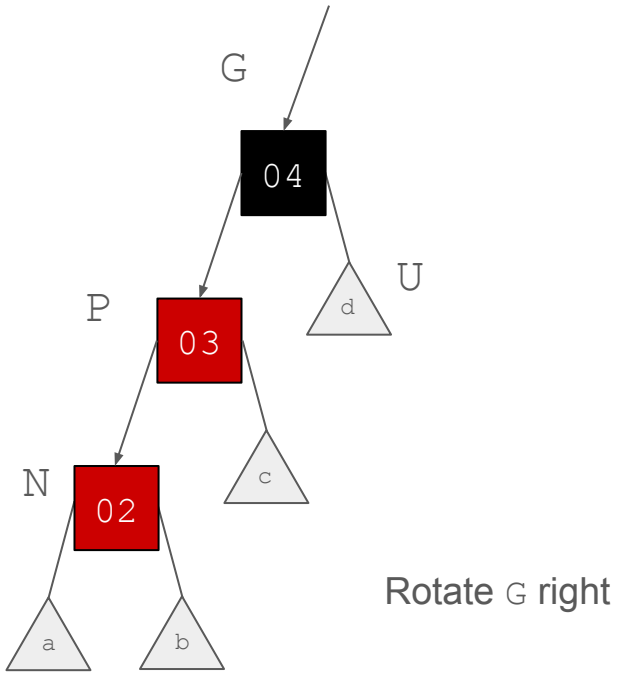
```rust
pub struct Node<K, V>
where
    K: Ord,
{
    pub key: K,
    pub value: V,
    pub color: Color,
    right: Option<Box<Node<K, V>>>,
    left: Option<Box<Node<K, V>>>,
}
```



Red-Black Trees: *Rust-Specific Stuff*

Rotations are harder than usual…

- Remember that you cannot have "partially constructed types"
- Making a temporary variable to perform the pointer swap won't work…



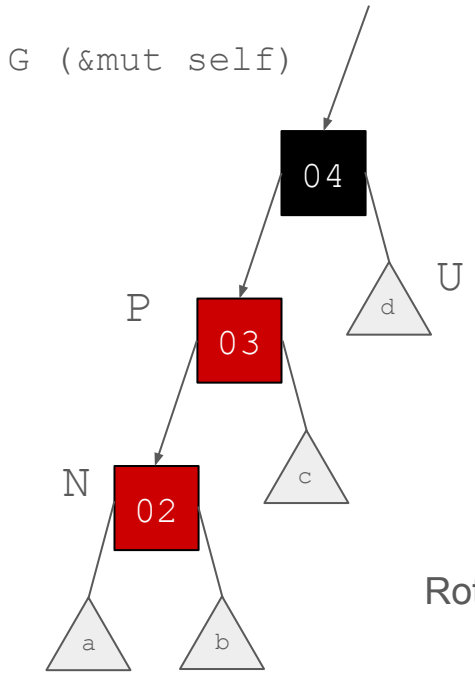
Rotate G right

Drawing subtrees now…

Red-Black Trees: *Rust-Specific Stuff*

Rotations are harder than usual…

1. Does `self.left` exist?
   a. If not, then we can't rotate → no-op
   b. If so, then we can rotate!

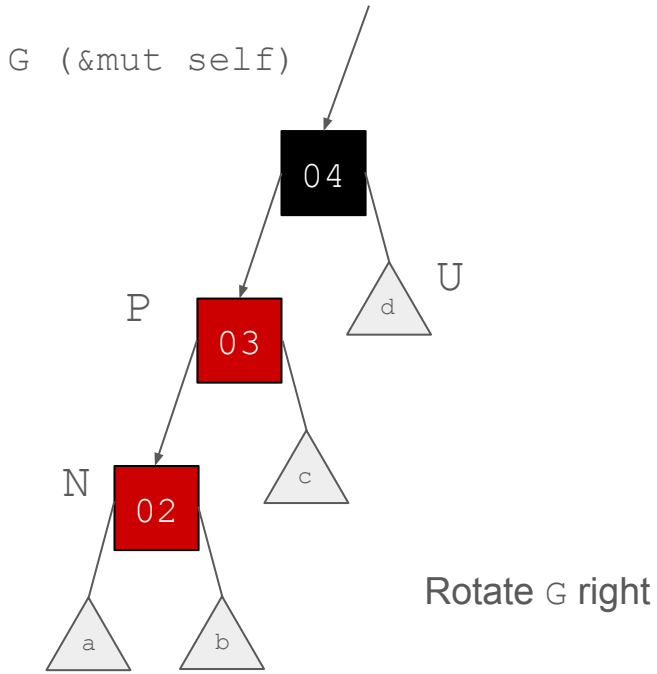


Rotate `G` right

Red-Black Trees: *Rust-Specific Stuff*

Rotations are harder than usual…

1. Does `self.left` exist?
2. Extract `self.left.left` and `self.left.right`
   a. Need to pull the value out of the `Option`, leaving a `None` in their place*



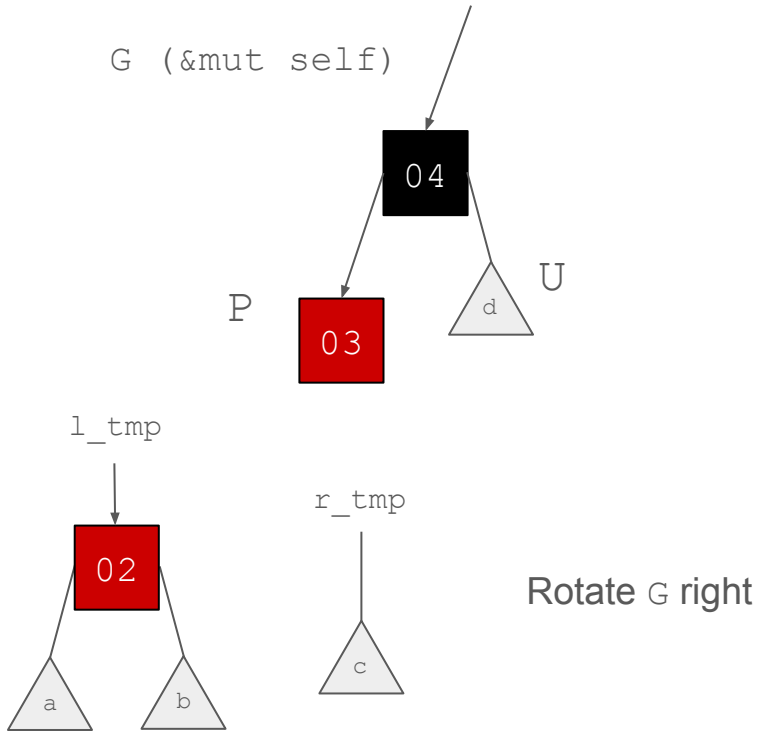
Rotate `G` right

* README has some helpful functions to do this

Red-Black Trees: *Rust-Specific Stuff*

Rotations are harder than usual…

1. Does `self.left` exist?
2. Extract `self.left.left` and `self.left.right`
   a. Need to pull the value out of the `Option`, leaving a `None` in their place*



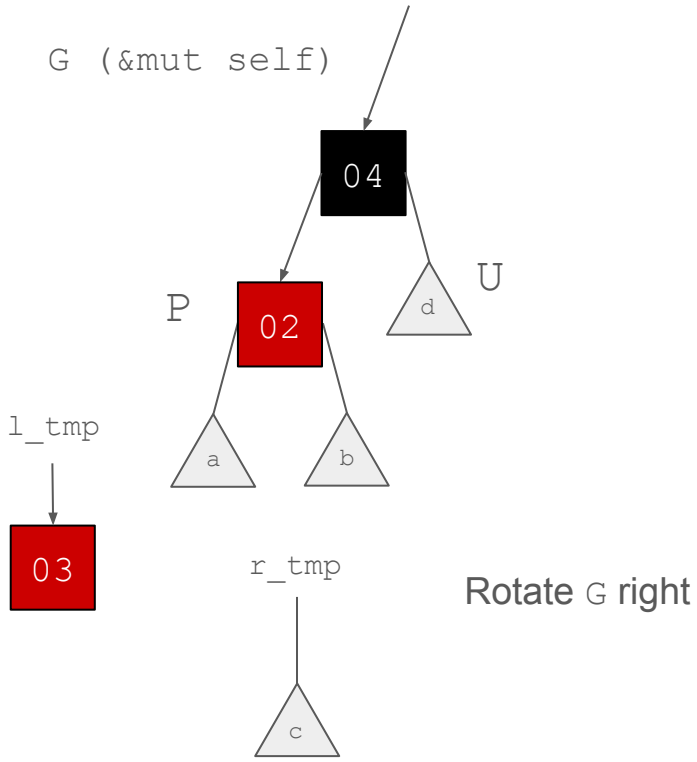
Rotate `G` right

* README has some helpful functions to do this

Red-Black Trees: *Rust-Specific Stuff*

Rotations are harder than usual…

1. Does `self.left` exist?
2. Extract `self.left.left` and `self.left.right`
3. Replace `P` contents with `l_tmp`

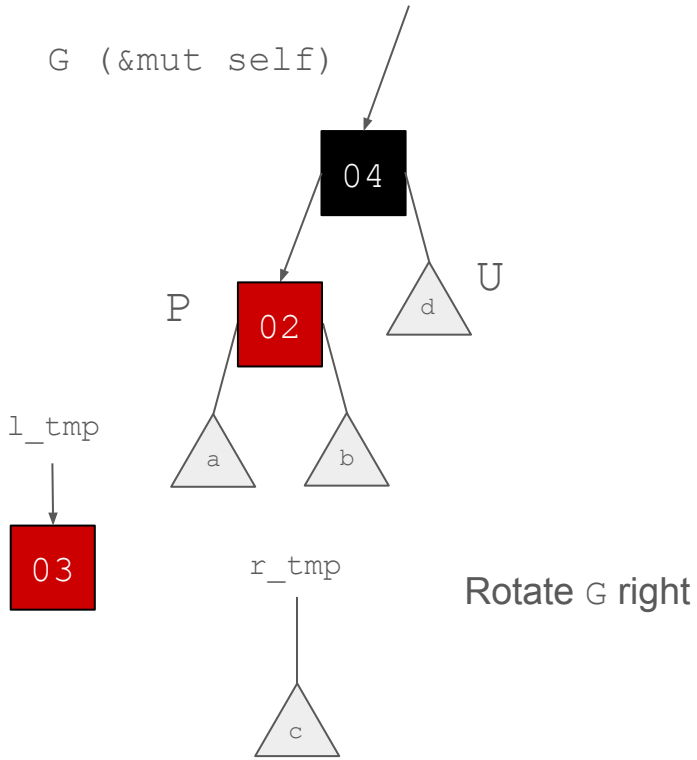


Rotate `G` right

Red-Black Trees: *Rust-Specific Stuff*

Rotations are harder than usual…

1. Does `self.left` exist?
2. Extract `self.left.left` and `self.left.right`
3. Replace `P` contents with `l_tmp`
4. Swap contents of `self` and `l_tmp`

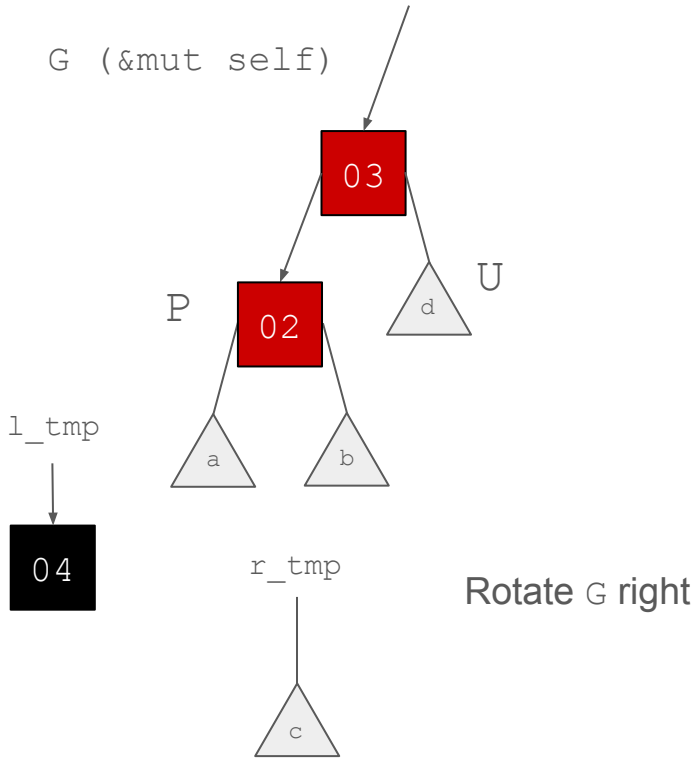


Rotate `G` right

Red-Black Trees: *Rust-Specific Stuff*

Rotations are harder than usual…

1. Does `self.left` exist?
2. Extract `self.left.left` and `self.left.right`
3. Replace `P` contents with `l_tmp`
4. Swap contents of `self` and `l_tmp`

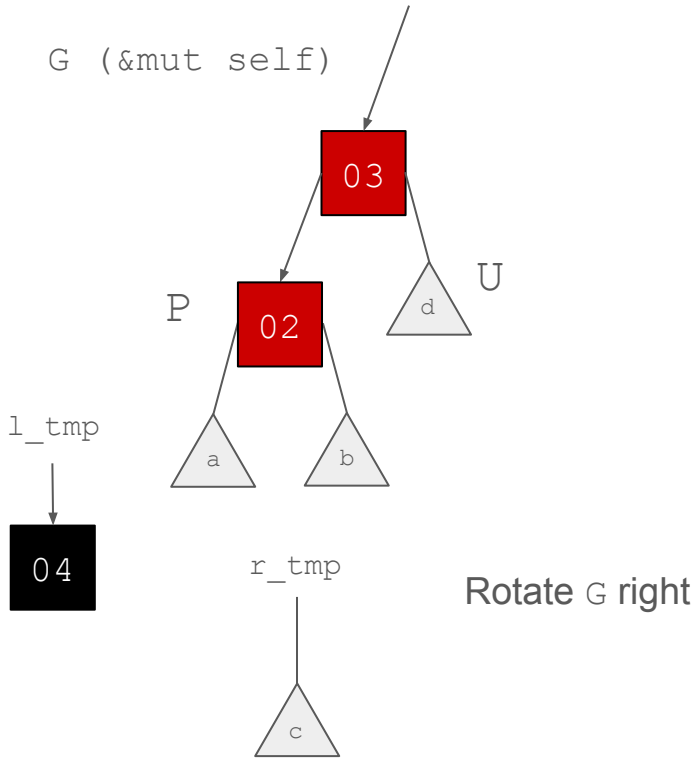


Rotate `G` right

Red-Black Trees: *Rust-Specific Stuff*

Rotations are harder than usual…

1. Does `self.left` exist?
2. Extract `self.left.left` and `self.left.right`
3. Replace `P` contents with `l_tmp`
4. Swap contents of `self` and `l_tmp`
5. Extract `self.right`

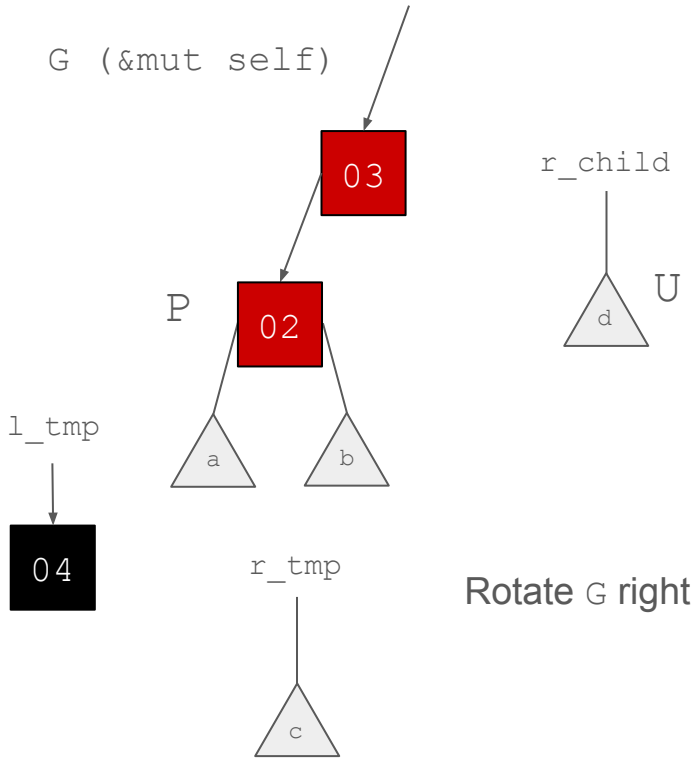


Rotate `G` right

Red-Black Trees: *Rust-Specific Stuff*

Rotations are harder than usual…

1. Does `self.left` exist?
2. Extract `self.left.left` and `self.left.right`
3. Replace `P` contents with `l_tmp`
4. Swap contents of `self` and `l_tmp`
5. Extract `self.right`

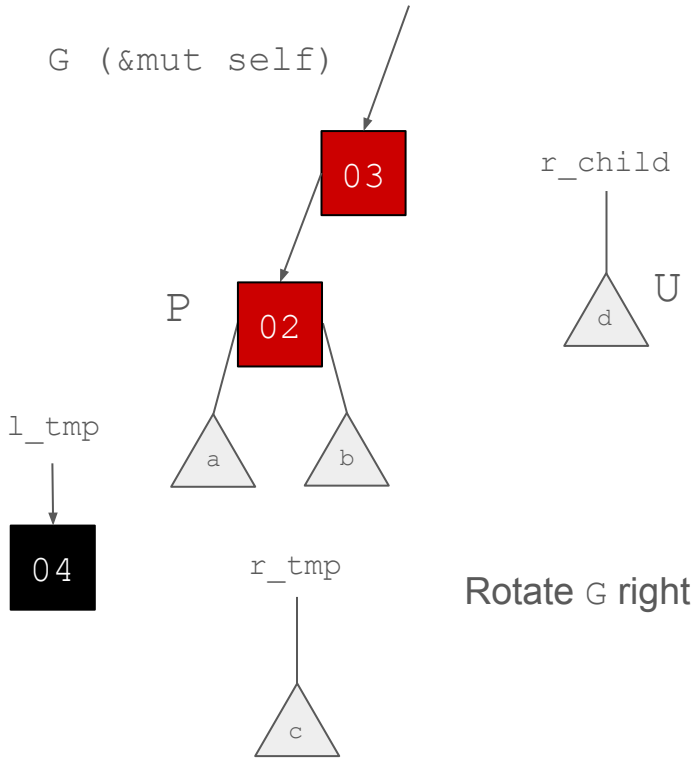


Rotate `G` right

Red-Black Trees: *Rust-Specific Stuff*

Rotations are harder than usual…

1. Does `self.left` exist?
2. Extract `self.left.left` and `self.left.right`
3. Replace `P` contents with `l_tmp`
4. Swap contents of `self` and `l_tmp`
5. Extract `self.right`
6. Construct new right child

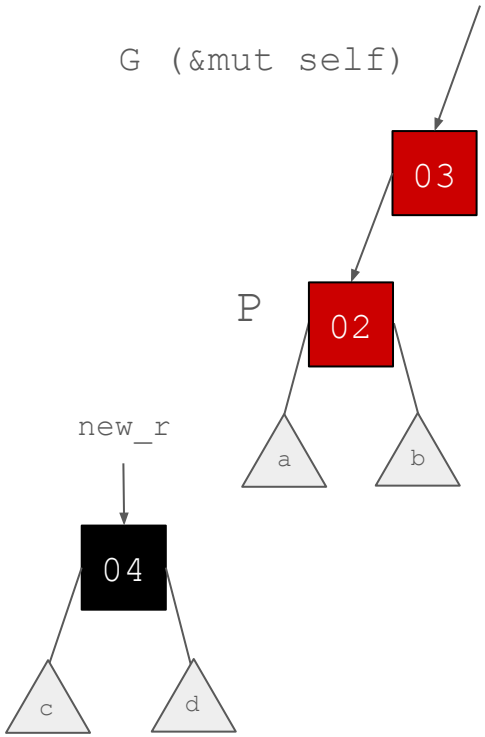


Rotate `G` right

Red-Black Trees: *Rust-Specific Stuff*

Rotations are harder than usual…

1. Does `self.left` exist?
2. Extract `self.left.left` and `self.left.right`
3. Replace `P` contents with `l_tmp`
4. Swap contents of `self` and `l_tmp`
5. Extract `self.right`
6. Construct new right child

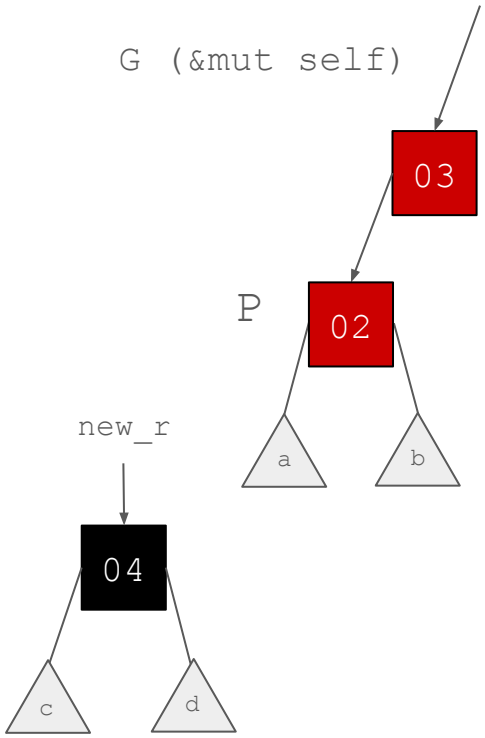


Rotate `G` right

Red-Black Trees: *Rust-Specific Stuff*

Rotations are harder than usual…

1. Does `self.left` exist?
2. Extract `self.left.left` and `self.left.right`
3. Replace `P` contents with `l_tmp`
4. Swap contents of `self` and `l_tmp`
5. Extract `self.right`
6. Construct new right child
7. Attach new right child

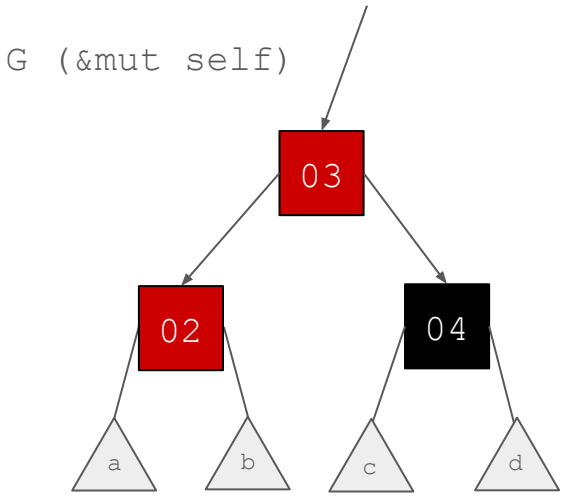


Rotate `G` right

Red-Black Trees: *Rust-Specific Stuff*

Rotations are harder than usual…

1. Does `self.left` exist?
2. Extract `self.left.left` and `self.left.right`
3. Replace `P` contents with `l_tmp`
4. Swap contents of `self` and `l_tmp`
5. Extract `self.right`
6. Construct new right child
7. Attach new right child



…and we are done!

Red-Black Trees: *Rust-Specific Stuff*

We give you an `for_each` which executes a function `f` on each node.

```rust
fn for_each(&self, order: &Traversal, f: &dyn Fn(&Node<K, V>)) {
    match order {
        Traversal::Preorder => {
            f(self);
            if let Some(left) = &self.left {
                left.for_each(order, f);
            }
            if let Some(right) = &self.right {
                right.for_each(order, f);
            }
        }
         ...
    }
}
```

Really helpful for writing tests, also look at the `check_tree` helper function to check invariants!